

程 序 源 码

软件名称：历史战争知识图谱构建与智能问答系统

软件版本：V1.0

第一部分：后端核心源码（Python）

backend/app.py

说明：Flask 应用主入口

```
"""
Flask 应用主入口 功能：提供 RESTful API 接口 - 用户认证：/api/login, /api/sign_in - 知识图谱查询：
/search_name_kg
- 节点管理：/create_node, /update_node, /delete_node - 智能问答：/api/ai/inference
数据流向：前端 → SQLite(主存储) → Neo4j(可视化)
"""

import json
import os
import time
import uuid
import atexit
import re
from collections import Counter

# ===== Flask 核心模块 =====
from flask import Flask, request, jsonify, g, Response
from flask_cors import CORS
from sqlalchemy.pool import NullPool
from sqlalchemy import func, text

# ===== 自定义模块 =====
from db_utils import DbUtil
from jwt_util import decode, encode
from model_search import neo4j_db
from inference.rule_llm_integration import DYNASTY_SCOPE_MAP
from models import (
    Event, Place, Organization, Person, EventEventRelation, EventOrganizationRel,
    EventPlaceRelation, EventPersonRelation, )

# ===== 创建 Flask 应用 =====
app = Flask(__name__) CORS(app) neo4j_db_handle = neo4j_db() shared_entity_extractor = None
shared_rule_llm_integration = None user_id = None

# ===== 数据库配置 =====
APP_PATH = os.path.dirname(__file__)
app.config['SQLALCHEMY_DATABASE_URI'] = f'sqlite:/// {APP_PATH}/database'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SQLALCHEMY_ENGINE_OPTIONS'] = { 'poolclass': NullPool, 'connect_args': {
    'check_same_thread': False, 'timeout': 30 } }
# 初始化关系型数据库（包含同步管理器初始化）
DbUtil.init_app(app)

def backfill_event_place_relation_evidence():
    """从当前 processed JSON 快速回填关系证据，不覆盖已有人工值。"""
    from models import db
```

```

def safe(value):
    if value is None:
        return ""
    return str(value).strip()

rel_path = os.path.join(APP_PATH, "data", "processed", "事件-地点关系表_event_place_relations.json")
if not os.path.exists(rel_path):
    return

try:
    with open(rel_path, "r", encoding="utf-8") as handle:
        rows = json.load(handle)
except Exception as exc:
    print(f"读取事件-地点关系证据失败: {exc}")
    return

evidence_index = {}
for item in rows:
    event_name = safe(item.get("EventName"))
    rel_type = safe(item.get("relation") or item.get("relations"))
    place_name = safe(item.get("modern_name") or item.get("geo_name") or item.get("PlaceName"))
    evidence = safe(item.get("evidence") or item.get("source_text"))
    if event_name and rel_type and place_name and evidence:
        evidence_index[(event_name, rel_type, place_name)] = evidence

if not evidence_index:
    return

updated = 0
for relation in EventPlaceRelation.query.all():
    if safe(getattr(relation, "evidence", "")):
        continue
    candidates = [
        (relation.event_name, relation.relation_type, relation.modern_name),
        (relation.event_name, relation.relation_type, relation.place_name),
    ]
    evidence = next((evidence_index.get(tuple(safe(value) for value in key)) for key in candidates if
evidence_index.get(tuple(safe(value) for value in key))), "")
    if evidence:
        relation.evidence = evidence
        relation.source_type = relation.source_type or "extraction"
        relation.confidence = relation.confidence or "medium"
        updated += 1

if updated:
    db.session.commit()
    print(f"已回填事件-地点关系证据 {updated} 条")

```

启用 WAL 模式

```

with app.app_context():
    from models import db

    try:
        db.session.execute(text("PRAGMA journal_mode=WAL;"))
        db.session.execute(text("PRAGMA synchronous=NORMAL;"))
        ensure_event_place_relation_metadata_columns()
        ensure_place_coordinate_metadata_columns() backfill_event_place_relation_evidence()
        db.session.commit()
        print("☑ SQLite WAL 模式已启用")
    except Exception as e:
        print(f"⚠ 设置 WAL 模式失败: {e}")

# 历史地名词典
historical_places = [] DYNASTY_DISPLAY_ORDER = [ "夏", "商", "西周", "东周", "春秋", "战国", "秦朝", "汉朝",
"西汉", "东汉", "三国", "魏国", "蜀国", "吴国", "晋朝", "西晋", "东晋", "南北朝", "隋朝", "唐朝", "五代十国",
"宋朝", "北宋",
"南宋", "辽朝", "金朝", "元朝", "明朝", "清朝", "民国", ] DYNASTY_ALIAS_MAP = { "夏朝": "夏", "商朝": "商",
"周朝": "东周", "秦": "秦朝", "汉": "汉朝", "隋": "隋朝", "唐": "唐朝", "宋": "宋朝", "辽": "辽朝", "金": "金朝",
"元": "元朝", "明": "明朝", "清": "清朝", } TYPE_LABELS = { "Event": "战争事件", "Place": "战争地点",
"Organization": "参战组织", "Person": "历史人物", } TYPE_ALIASES = { "Event": "Event", "战争事件": "Event",
"事件": "Event", "Place": "Place", "战争地点": "Place", "地点": "Place",
"Organization": "Organization", "参战组织": "Organization", "势力组织": "Organization",
"组织": "Organization", "Person": "Person", "历史人物": "Person", "人物": "Person", }
SOURCE_TYPE_LABELS = { "graph": "来自图谱", "rule": "规则推理", "llm": "大模型补充", }
PROVINCE_CENTROIDS = { "北京": (116.40, 39.90), "天津": (117.20, 39.12), "河北": (114.48, 38.03),
"山西": (112.55, 37.87), "内蒙古": (111.67, 40.82), "辽宁": (123.43, 41.80), "吉林": (125.32, 43.90),
"黑龙江": (126.63, 45.75), "上海": (121.47, 31.23), "江苏": (118.78, 32.04), "浙江": (120.15, 30.28),
"安徽": (117.27, 31.86), "福建": (119.30, 26.08), "江西": (115.89, 28.68), "山东": (117.00, 36.65),
"河南": (113.62, 34.75), "湖北": (114.30, 30.60), "湖南": (112.98, 28.20), "广东": (113.27, 23.13),
"广西": (108.32, 22.82), "海南": (110.35, 20.02), "重庆": (106.55, 29.57), "四川": (104.07, 30.67),
"贵州": (106.71, 26.57), "云南": (102.71, 25.04), "西藏": (91.13, 29.65), "陕西": (108.95, 34.27),
"甘肃": (103.82, 36.07), "青海": (101.78, 36.62), "宁夏": (106.27, 38.47), "新疆": (87.62, 43.82),
"香港": (114.17, 22.28), "澳门": (113.54, 22.19), "台湾": (121.51, 25.04), } CITY_CENTROIDS = {
"西安": (108.95, 34.27), "洛阳": (112.44, 34.66), "安阳": (114.39, 36.10), "新乡": (113.93, 35.31),
"焦作": (113.24, 35.22), "运城": (111.01, 35.02), "渭南": (109.50, 34.50), "长治": (113.12, 36.20),
"宝鸡": (107.23, 34.36), "咸阳": (108.70, 34.33), "郑州": (113.62, 34.75), "开封": (114.30, 34.80),
"邯郸": (114.54, 36.63), "邢台": (114.49, 37.06), "濮阳": (115.03, 35.76), "商丘": (115.65, 34.44),
"平顶山": (113.19, 33.77), "南阳": (112.53, 32.99), "十堰": (110.78, 32.65), "武汉": (114.31, 30.52),
"成都": (104.07, 30.67), "太原": (112.55, 37.87), "北京": (116.40, 39.90), "南京": (118.78, 32.04),
"杭州": (120.15, 30.28), } HISTORICAL_REGION_CENTROIDS = { "汉水": (112.15, 32.05),
"汉水流域": (112.35, 31.75), "荆楚": (112.20, 30.35), "湖北长江流域": (112.30, 30.40),
"淮河流域": (116.80, 32.90), "东夷": (118.40, 35.20), "东夷(山东": (117.00, 36.65),
"江苏一带)": (118.78, 32.04), "西北地区": (108.95, 34.27), "徐夷地区": (117.20, 34.25),
"淮夷地区": (116.80, 32.90), "徐戎": (118.20, 33.45), "犬丘": (107.25, 34.35), "骊山": (109.21, 34.37), }
ROUTE_RELATION_PRIORITY = { "出发地": 1, "驻防地": 2, "途经地": 3, "补给地": 4, "指挥所": 5, "主战场": 6,
"次要战场": 7, "战略要地": 8, "议和地点": 9, "目的地": 10, }

# ===== 初始化函数 =====

```

```

def build_map_overview(keyword='', dynasty=''):
    place_query = Place.query
    if keyword:
        place_query = place_query.filter( (Place.name.contains(keyword)) |
                                           (Place.modern_name.contains(keyword)) | (Place.province.contains(keyword)) |
                                           (Place.city.contains(keyword)) | (Place.district.contains(keyword)) )
    if dynasty:
        filter_values = _dynasty_filter_values(dynasty)
        if filter_values:
            place_query = place_query.filter(Place.dynasty.in_(filter_values))

    places = place_query.limit(300).all() allowed_place_ids = {place.id for place in places}
    points = [] dynasty_counter = Counter() all_event_ids = set() mappable_event_ids = set()
    event_point_map = {} unmapped_places = []
    for place in places:
        coord_result = _resolve_place_coordinates(place)
        coords = coord_result["coords"] if coord_result else None
        related_rows = EventPlaceRelation.query.filter(EventPlaceRelation.place_id == place.id).all()
        event_ids = [] route_segments = [] related_events = [] entity_counter = {
            "persons": {}, "organizations": {}, "related_events": {}, }
        for row in related_rows:
            event = Event.query.get(row.event_id)
            if not event:
                continue
            if dynasty and _normalize_dynasty_name(event.dynasty) != _normalize_dynasty_name(dynasty):
                continue all_event_ids.add(event.id) event_ids.append(event.id)
            participant_bundle = _event_participants(event.id)
            event_brief = _event_brief(event) event_brief["entities"] = participant_bundle
            related_events.append(event_brief)
            dynasty_counter[_normalize_dynasty_name(event.dynasty) or "未标注"] += 1
            route_segments.append({ "event_id": event.id, "event_name": event.name,
                                    "relation_type": row.relation_type, "from": event.aggressor or "未知发起方",
                                    "to": place.modern_name or place.name, "evidence": row.evidence,
                                    "source_type": row.source_type or "extraction",
                                    "confidence": row.confidence or ("medium" if _safe_text(row.evidence) else ""),
                                })
        if coords:
            mappable_event_ids.add(event.id) event_point_map.setdefault(event.id, {
                "event": event_brief, "nodes": [], })
            event_point_map[event.id]["nodes"].append({ "place_id": place.id,
                                                         "name": place.modern_name or place.name, "place_name": place.name,
                                                         "modern_name": place.modern_name,
                                                         "coord_hint": " / ".join([item for item in [place.province, place.city, place.district]
                                                                 if _safe_text(item)]),
                                                         "coords": [coords[0], coords[1]],
                                                         "coord_source": coord_result["coord_source"],
                                                         "coord_source_label": coord_result["coord_source_label"],
                                                         "coord_confidence": coord_result["coord_confidence"],

```

```

        "coord_matched_text": coord_result["coord_matched_text"],
        "coord_note": coord_result["coord_note"],
        "relation_type": row.relation_type, "evidence": row.evidence,
        "source_type": row.source_type or "extraction",
        "confidence": row.confidence or ("medium" if _safe_text(row.evidence) else ""),
        "priority": ROUTE_RELATION_PRIORITY.get(row.relation_type, 99),
    })

    for person in participant_bundle["persons"]:
        entity_counter["persons"][person["id"]] = person
    for org in participant_bundle["organizations"]:
        entity_counter["organizations"][org["id"]] = org
    for related_event in participant_bundle["related_events"]:
        entity_counter["related_events"][related_event["id"]] = related_event

    if not event_ids:
        continue

    if not coords:
        unmapped_places.append({ "place_id": place.id, "place_name": place.name,
            "modern_name": place.modern_name, "dynasty": place.dynasty,
            "coord_hint": " / ".join([item for item in [place.province, place.city, place.district] if
            _safe_text(item)]),
            "event_count": len(set(event_ids)), "events": related_events[:6],
            "reason": "缺少可解析坐标", "coord_source": "unresolved", "coord_source_label": "未解析",
            "coord_confidence": "", "coord_note": "地点表无显式坐标, 且现代地名、行政区和内置历史区域表
均未命中",
            "detail_route": f"/knowledge/entity-detail?type=Place&id={place.id}",
        })

    persons = list(entity_counter["persons"].values())[:10]
    organizations = list(entity_counter["organizations"].values())[:10]
    related_event_entities = list(entity_counter["related_events"].values())[:10]
    points.append({ "place_id": place.id, "place_name": place.name,
        "modern_name": place.modern_name, "dynasty": place.dynasty, "province": place.province,
        "city": place.city, "district": place.district,
        "specific_location": place.specific_location, "event_count": len(set(event_ids)),
        "label_name": place.modern_name or place.name,
        "coord_hint": " / ".join([item for item in [place.province, place.city, place.district] if
        _safe_text(item)]),
        "compare_name": f"{place.name} / {place.modern_name}" if _safe_text(place.modern_name) and
place.modern_name != place.name else place.name,
        "longitude": coords[0] if coords else None,
        "latitude": coords[1] if coords else None,
        "has_real_coord": bool(coords),
        "coord_source": coord_result["coord_source"] if coord_result else "unresolved",
        "coord_source_label": coord_result["coord_source_label"] if coord_result else "未解析",
        "coord_confidence": coord_result["coord_confidence"] if coord_result else "",
        "coord_matched_text": coord_result["coord_matched_text"] if coord_result else "",
        "coord_note": coord_result["coord_note"] if coord_result else "",

```

```

        "event_ids": sorted(set(event_ids)), "events": related_events[:8], "entities": {
            "persons": persons, "organizations": organizations,
            "related_events": related_event_entities, }, "routes": route_segments[:6],
            "detail_route": f"/knowledge/entity-detail?type=Place&id={place.id}",
        })

points.sort(key=lambda item: (-item["event_count"], item["label_name"]))
unmapped_places.sort(key=lambda item: (-item["event_count"], item["place_name"]))

event_points = []
for event_id, item in event_point_map.items():
    nodes = sorted(item["nodes"], key=lambda node: (node["priority"], node["name"]))
    if not nodes:
        continue
    primary_node = nodes[0]
    event = item["event"]
    event_points.append({
        "event_id": event_id, "event_name": event["name"], "dynasty": event.get("dynasty"),
        "event_type": event.get("event_type"), "place_id": primary_node["place_id"],
        "place_name": primary_node["place_name"],
        "modern_name": primary_node["modern_name"], "label_name": event["name"],
        "longitude": primary_node["coords"][0], "latitude": primary_node["coords"][1],
        "coord_hint": primary_node["coord_hint"],
        "coord_source": primary_node.get("coord_source"),
        "coord_source_label": primary_node.get("coord_source_label"),
        "coord_confidence": primary_node.get("coord_confidence"),
        "coord_matched_text": primary_node.get("coord_matched_text"),
        "coord_note": primary_node.get("coord_note"),
        "relation_type": primary_node["relation_type"],
        "evidence": primary_node.get("evidence"),
        "source_type": primary_node.get("source_type"),
        "confidence": primary_node.get("confidence"), "related_place_count": len(nodes),
        "has_route": len({f"{node['coords']}[0]}-{node['coords']}[1]" for node in nodes}) >= 2,
        "detail_route": event.get("detail_route"),
    })

event_points.sort(key=lambda item: (not item["has_route"], item["event_name"]))

hot_routes = []
for item in points[:30]:
    for route in item.get("routes", []):
        hot_routes.append({
            "event_id": route["event_id"],
            "event_name": route["event_name"], "relation_type": route["relation_type"],
            "from_label": route["from"], "to_label": route["to"],
            "place_name": item["place_name"], })
        route_lines = []
        event_route_source = {}
        event_query = Event.query

if dynasty:
    filter_values = _dynasty_filter_values(dynasty)
    if filter_values:
        event_query = event_query.filter(Event.dynasty.in_(filter_values))

for event in event_query.limit(500).all():
    route_rows = EventPlaceRelation.query.filter(EventPlaceRelation.event_id == event.id).all()

```

```

for row in route_rows:
    if allowed_place_ids and row.place_id not in allowed_place_ids:
        continue
    place = Place.query.get(row.place_id)
    if not place:
        continue
    coord_result = _resolve_place_coordinates(place)
    coords = coord_result["coords"] if coord_result else None
    if not coords:
        continue
    event_route_source.setdefault(event.id, { "event_id": event.id,
        "event_name": event.name, "nodes": [], })
    event_route_source[event.id]["nodes"].append({
        "name": place.modern_name or row.modern_name or place.name or row.place_name,
        "place_name": place.name or row.place_name,
        "coords": [coords[0], coords[1]],
        "coord_source": coord_result["coord_source"],
        "coord_source_label": coord_result["coord_source_label"],
        "coord_confidence": coord_result["coord_confidence"],
        "coord_matched_text": coord_result["coord_matched_text"],
        "coord_note": coord_result["coord_note"], "relation_type": row.relation_type,
        "evidence": row.evidence, "source_type": row.source_type or "extraction",
        "confidence": row.confidence or ("medium" if _safe_text(row.evidence) else ""),
        "priority": ROUTE_RELATION_PRIORITY.get(row.relation_type, 99),
    })

for route_item in event_route_source.values():
    nodes = sorted(route_item["nodes"], key=lambda item: (item["priority"], item["name"]))
    seen_names = set()
    deduped = []
    for node in nodes:
        key = f"{node['coords'][0]}-{node['coords'][1]}"
        if key in seen_names:
            continue
        seen_names.add(key)
        deduped.append(node)
    if len(deduped) < 2:
        continue
    for index in range(len(deduped) - 1):
        route_lines.append({ "event_id": route_item["event_id"],
            "event_name": route_item["event_name"], "from_name": deduped[index]["name"],
            "to_name": deduped[index + 1]["name"],
            "coords": [deduped[index]["coords"], deduped[index + 1]["coords"]],
            "relation_chain": f"{deduped[index]['relation_type']} → {deduped[index +
1]['relation_type']}",
            "relation_steps": [ { "place_name": deduped[index]["place_name"],
                "name": deduped[index]["name"],
                "relation_type": deduped[index]["relation_type"],
                "evidence": deduped[index].get("evidence"),
                "source_type": deduped[index].get("source_type"),

```

```

        "confidence": deduped[index].get("confidence"),
        "coord_source_label": deduped[index].get("coord_source_label"),
        "coord_confidence": deduped[index].get("coord_confidence"),
        "coord_note": deduped[index].get("coord_note"), }, {
        "place_name": deduped[index + 1]["place_name"],
        "name": deduped[index + 1]["name"],
        "relation_type": deduped[index + 1]["relation_type"],
        "evidence": deduped[index + 1].get("evidence"),
        "source_type": deduped[index + 1].get("source_type"),
        "confidence": deduped[index + 1].get("confidence"),
        "coord_source_label": deduped[index + 1].get("coord_source_label"),
        "coord_confidence": deduped[index + 1].get("coord_confidence"),
        "coord_note": deduped[index + 1].get("coord_note"), }, ],
        "route_source": "inferred_by_relation_order",
        "route_source_label": "按关系类型推断", "route_confidence": "medium", })

route_event_ids = {item["event_id"] for item in route_lines}
single_point_event_ids = mappable_event_ids - route_event_ids
coord_source_counter = Counter(item.get("coord_source_label") or "未解析" for item in points if
item.get("has_real_coord"))
low_confidence_places = [
    item for item in points
    if item.get("has_real_coord") and item.get("coord_confidence") in {"low", "低"}
]

return {
    "summary": { "place_count": len(points), "total_events": len(all_event_ids),
    "covered_events": len(all_event_ids), "mappable_events": len(mappable_event_ids),
    "route_events": len(route_event_ids),
    "single_point_events": len(single_point_event_ids),
        "unmapped_events": len(all_event_ids - mappable_event_ids),
        "dynasty_count": len([key for key in dynasty_counter.keys() if key]),
        "real_coord_points": len([item for item in points if item.get("has_real_coord")]),
        "mappable_places": len([item for item in points if item.get("has_real_coord")]),
        "unmapped_places": len(unmapped_places),
        "low_confidence_places": len(low_confidence_places),
    },
    "dynasties": sorted([key for key in dynasty_counter.keys() if key]),
    "points": points[:120], "place_points": points[:120],
    "event_points": event_points[:160], "unmapped_places": unmapped_places[:120],
    "dynasty_distribution": [{"name": key, "value": value} for key, value in
dynasty_counter.most_common(12)],
        "coord_source_distribution": [{"name": key, "value": value} for key, value in
coord_source_counter.most_common()],
        "routes": hot_routes[:80], "route_lines": route_lines[:120], }

def ai_inference_stream():
    """智能问答推理接口 - SSE 流式输出 版本 处理流程: 1. 实体提取: 从问题中识别历史实体 2. 图谱查询: 从 Neo4j
    查询实体关系 3. 规则推理: 应用规则推导隐含关系
    4. 大模型生成: 流式生成自然语言回答 响应: SSE (Server-Sent Events) 流 """
    try:

```

```

data = request.get_json()
if not data:
    return jsonify({'success': False, 'error': '请求参数不能为空'}), 400

user_question = data.get('question', '')
if not user_question or len(user_question.strip()) == 0:
    return jsonify({'success': False, 'error': '问题不能为空'}), 400

request_id = str(uuid.uuid4())[:8]
print(f"[{request_id}] 收到 SSE 推理请求: '{user_question}'")

# 获取推理引擎和实体提取器
if not hasattr(g, 'rule_llm_integration') or not hasattr(g, 'entity_extractor'):
    try:
        from inference.rule_llm_integration import RuleLLMIntegration
        from entity_extract.extractor import Extractor
        g.rule_llm_integration = RuleLLMIntegration(
            rule_file_path='rules/rule_base.json', model_name='deepseek-rl:7b',
            max_depth=30 ) g.entity_extractor = Extractor()
    except Exception as init_err:
        print(f"[{request_id}] 初始化失败: {str(init_err)}")
        return jsonify({'success': False, 'error': '系统初始化失败'}), 500

# 在生成器外部捕获 Flask 上下文对象，避免在生成器内访问 g
rule_engine = g.rule_llm_integration
entity_ext = g.entity_extractor

def generate():
    """SSE 生成器"""
    try:
        # 发送开始信号
        yield f"data: {json.dumps({'status': 'start', 'request_id': request_id})}\n\n"

        # ===== 步骤 1: 实体提取 =====
        yield f"data: {json.dumps({'status': 'extracting', 'message': '正在识别实体...'})}\n\n"

        start_time = time.time()

        # 检测问题类型
        dynasty_scope = rule_engine._detect_dynasty_event_scope(user_question)
        if dynasty_scope:
            extracted_entities = [] query_entities = [dynasty_scope["display"]]
            event_detail_query = False participant_event_query = False
        else:
            extracted_entities = entity_ext.extract_entities(user_question)
            query_entities = rule_engine._extract_query_entities(user_question, extracted_entities)
            event_detail_query = rule_engine._looks_like_event_detail_query(user_question,
extracted_entities)
            participant_event_query = rule_engine._looks_like_participant_event_query(user_question)

```

```

# 过滤实体
if dynasty_scope:
    entities = query_entities.copy()
else:
    entities = rule_engine._filter_relevant_entities(extracted_entities, user_question,
neo4j_db_handle)

extract_time = time.time() - start_time
print(f"[{request_id}] 实体提取完成, 耗时: {extract_time:.2f}秒, 实体: {entities}")

# 发送实体信息
yield f"data: {json.dumps({'status': 'entities', 'entities': entities, 'query_entities':
query_entities, 'extract_time': round(extract_time, 2)}})\n\n"

if not entities:
    yield f"data: {json.dumps({'status': 'done', 'answer': '抱歉, 无法从问题中识别出实体.',
'kg_data': {'nodes': [], 'lines': []}})}\n\n"
    return

# ===== 步骤2: 查询知识图谱 =====
yield f"data: {json.dumps({'status': 'querying', 'message': '正在查询知识图谱...'})}\n\n"

query_start = time.time()

# 查询实体信息
all_entity_info = []
entity_info_map = {}

if dynasty_scope:
    all_entity_info = rule_engine._get_dynasty_event_infos(neo4j_db_handle,
dynasty_scope["dynasties"])
    print(f"[{request_id}] 朝代查询: {dynasty_scope['dynasties']}, 找到
{len(all_entity_info)} 条事件")
    for info in all_entity_info:
        entity_info_map[info['id']] = info
    elif event_detail_query:
        event_names = [entity for entity in entities if any(suffix in entity for suffix in ["之
战", "战役", "起义", "叛乱"])]
        all_entity_info = rule_engine._get_event_detail_infos(neo4j_db_handle, event_names or
entities)

        for info in all_entity_info:
            entity_info_map[info['id']] = info
    elif participant_event_query:
        participant_names = query_entities or entities
        all_entity_info = rule_engine._get_participant_event_infos(neo4j_db_handle,
participant_names)

        for info in all_entity_info:
            entity_info_map[info['id']] = info

```

```

else:
    for entity in entities:
        query = f""" MATCH (n) WHERE n.name = '{entity}' RETURN n LIMIT 1 """
        results = neo4j_db_handle.graph.run(query).data()
        if results:
            node = results[0]['n'] info = { 'id': node.identity,
            'name': node['name'],
            'type': list(node.labels)[0] if node.labels else '',
            'properties': {k: v for k, v in node.items()} }
            all_entity_info.append(info) entity_info_map[info['id']] = info
        else:
            # 朝代别名回退：如“周朝”→查 DynastyName 为“西周”/“东周”的事件
            dynasty_values = DYNASTY_SCOPE_MAP.get(entity)
            if dynasty_values:
                dynasty_query = """ MATCH (n:Event)
                WHERE n.dynasty IN $dynasties RETURN n
                ORDER BY coalesce(n.start_date, ''), n.name LIMIT 20 """
                dynasty_results = neo4j_db_handle.graph.run(dynasty_query,
                dynasties=dynasty_values).data()
                for result in dynasty_results:
                    node = result['n'] info = { 'id': node.identity,
                    'name': node['name'],
                    'type': list(node.labels)[0] if node.labels else 'Event',
                    'properties': {k: v for k, v in node.items()} }
                    all_entity_info.append(info)
                    entity_info_map[info['id']] = info
                if dynasty_results:
                    print(f"[{request_id}] 朝代别名回退：'{entity}' → {dynasty_values}，找到 {len(dynasty_results)} 条事件")
                    continue

            # 模糊查询
            fuzzy_query = f"""
            MATCH (n)
            WHERE n.name CONTAINS '{entity}' OR '{entity}' CONTAINS n.name
            RETURN n
            LIMIT 5
            """
            fuzzy_results = neo4j_db_handle.graph.run(fuzzy_query).data()
            for result in fuzzy_results:
                node = result['n'] info = { 'id': node.identity,
                'name': node['name'],
                'type': list(node.labels)[0] if node.labels else '',
                'properties': {k: v for k, v in node.items()} }
                all_entity_info.append(info) entity_info_map[info['id']] = info
            if not all_entity_info:
                yield f"data: {json.dumps({'status': 'done', 'answer': f'抱歉，在知识库中找不到与 {entities} 相关的实体信息。', 'kg_data': {'nodes': [], 'lines': []})}\n\n"
            return

```

```

# 查询实体关系
all_relationships = []
for info in all_entity_info:
    relationships = rule_engine.get_entity_relationships(info['id'], neo4j_db_handle)
    all_relationships.extend(relationships)

# 搜索实体间路径
all_paths = []
if len(all_entity_info) >= 2 and not (dynasty_scope or event_detail_query or
participant_event_query):
    processed_entity_pairs = set()
    for i in range(len(all_entity_info)):
        for j in range(i+1, len(all_entity_info)):
            entity1_id = all_entity_info[i]['id']
            entity2_id = all_entity_info[j]['id']
            if entity1_id == entity2_id:
                continue
            entity_pair = tuple(sorted([entity1_id, entity2_id]))
            if entity_pair in processed_entity_pairs:
                continue
            processed_entity_pairs.add(entity_pair)
            paths = rule_engine.search_paths_between_entities(entity1_id, entity2_id,
neo4j_db_handle)

            all_paths.extend(paths)

# 应用推理规则
inferred_relationships = rule_engine.apply_inference_rules(all_relationships)
original_relationships = all_relationships.copy()
all_relationships.extend(inferred_relationships)
query_time = time.time() - query_start
print(f"[{request_id}] 图谱查询完成, 耗时: {query_time:.2f}秒")

# 发送查询结果
yield f"data: {json.dumps({'status': 'queried', 'entity_count': len(all_entity_info),
'relation_count': len(all_relationships), 'query_time': round(query_time, 2)}})\n\n"

# ===== 步骤 3: 流式生成回答 =====
yield f"data: {json.dumps({'status': 'generating', 'message': '正在生成回答...'})}\n\n"

generate_start = time.time()

# 构建提示词
answer_entities = entities
question_type = "general"
answer_metadata = {}
if dynasty_scope:
    question_type = "dynasty_event_list"
    answer_metadata["dynasty_scope"] = dynasty_scope

```

```

        answer_entities = [f"{dynasty_scope['display']}" (仅限 DynastyName 属于 {'',
'.join(dynasty_scope['dynasties'])} 的战争事件)"]
    elif event_detail_query:
        question_type = "event_detail"
    elif participant_event_query:
        question_type = "participant_event_list"

# 构建提示词
if question_type == "dynasty_event_list":
    user_prompt = rule_engine._build_dynasty_event_list_prompt(
        question=user_question,
        dynasty_scope=answer_metadata.get("dynasty_scope", {}),
        event_info=all_entity_info
    )
elif question_type == "event_detail":
    user_prompt = rule_engine._build_event_detail_prompt(user_question, all_entity_info)
elif question_type == "participant_event_list":
    user_prompt = rule_engine._build_participant_event_prompt(user_question,
all_entity_info)
else:
    user_prompt = rule_engine._build_inference_prompt( question=user_question,
        entities=answer_entities, entity_info=all_entity_info,
        relationships=all_relationships, paths=all_paths )

# 设置 system 提示词
if question_type in {"dynasty_event_list", "participant_event_list"}:
    system_prompt = """你是一位中国历史战争知识图谱问答助手。当前任务是根据后端提供的结构化
事件清单回答枚举类问题。
必须完整覆盖清单中的每一条事件；不得自行删减、合并、补充清单外事件；不得把枚举题回答成单个“最准确答案”。"""
elif question_type == "event_detail":
    system_prompt = """你是一位中国历史战争知识图谱问答助手。当前任务是根据后端提供的结构化
事件记录回答事件详情问题。
只能使用记录中的字段作答；字段缺失时说明图谱未提供；不得补充图谱外史实。"""
else:
    system_prompt = """你是一位专业的中国历史战争研究与事件关系分析专家，精通中国历代战争事
件、参战势力、战役过程及其历史背景。
你的职责是： 1. 只回答用户提出的原始问题，不要重复或引用我给你的指令内容 2. 不要在回答中说“根据提供的信息”
或“基于图谱数据”等引导语
3. 所有回答必须基于知识图谱提供的事实，不要编造不存在的关系 4. 答案应直接、简洁，不要添加不必要的解释
重要：不要将指令或提示词本身视为用户问题的一部分。用户的原始问题已在提示词开头明确标出。"""

# 使用 ollama 流式调用
import ollama

full_answer = ""
try:
    stream = ollama.chat( model='deepseek-r1:7b', messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt} ], stream=True, keep_alive="10m",
        options={"temperature": 0.1} )
    for chunk in stream:

```

```

        if chunk['message']['content']:
            content = chunk['message']['content']
            full_answer += content
            # 发送内容块
            yield f"data: {json.dumps({'status': 'content', 'content': content})}\n\n"

    except Exception as llm_err:
        print(f"[{request_id}] LLM 调用失败: {str(llm_err)}")
        yield f"data: {json.dumps({'status': 'error', 'message': f'大模型调用失败: {str(llm_err)}'})}\n\n"
        return generate_time = time.time() - generate_start
        total_time = time.time() - start_time
        print(f"[{request_id}] 回答生成完成, 耗时: {generate_time:.2f}秒, 总耗时: {total_time:.2f}秒")

    # ===== 步骤4: 构建可视化数据 =====
    kg_data = rule_engine._convert_to_visual_data(all_entity_info, original_relationships,
all_paths)

    # 格式化关系上下文
    context = rule_engine._format_relations_for_context(original_relationships,
inferred_relationships)

    # 发送完成信号
    yield f"data: {json.dumps({'status': 'done', 'kg_data': kg_data, 'relations_text': context,
'entities': entities, 'query_entities': query_entities, 'process_time': round(total_time, 2)})}\n\n"

    except Exception as e:
        error_type = type(e).__name__
        error_msg = str(e)
        print(f"[{request_id}] SSE 处理出错: {error_type} - {error_msg}")
        import traceback
        traceback.print_exc()
        yield f"data: {json.dumps({'status': 'error', 'message': f'处理出错: {error_msg}'})}\n\n"

    return Response(generate(), mimetype='text/event-stream',
                    headers={'Cache-Control': 'no-cache', 'Connection': 'keep-alive',
                    'X-Accel-Buffering': 'no'})

except Exception as e:
    error_type = type(e).__name__
    error_msg = str(e)
    print(f"SSE 请求处理错误: {error_type} - {error_msg}")
    import traceback
    traceback.print_exc()
    return jsonify({'success': False, 'error': f'请求处理错误: {error_msg}'}), 500

if __name__ == "__main__":
    print("\n" + "=" * 60)
    print("🌐 启动历史地名知识图谱系统")

```

```

print("=" * 60)
print("📍 访问地址: http://localhost:5000")
print("💾 主存储: SQLite 关系型数据库")
print("📊 可视化: Neo4j 图数据库")
print("=" * 60 + "\n")

app.run(debug=True, port=5000, host='0.0.0.0')

```

backend/inference/rule_llm_integration.py

说明: 规则引擎与大模型集成

"""规则引擎与大模型集成推理模块 功能说明: - 规则引擎: 基于预定义规则 (rules/rule_base.json) 进行推理
- 大模型集成: 使用 deepseek-r1:7b 模型生成自然语言回答 - 知识图谱查询: 从 Neo4j 查询实体关系和路径 - 智能问答: 结合规则推理和大模型生成最终答案 处理流程:
1. 从问题中提取实体 2. 从 Neo4j 查询实体信息和关系 3. 应用规则推理生成隐含关系 4. 使用大模型生成最终回答 5. 返回回答和知识图谱可视化数据 """

```

import os
import json
import time
import ollama
import hashlib
from typing import List, Dict, Any, Optional
from collections import OrderedDict
from concurrent.futures import ThreadPoolExecutor, as_completed

def _lru_get(cache, key):
    if key not in cache:
        return None
    value = cache.pop(key)
    cache[key] = value
    return value

def _lru_set(cache, key, value, max_size=128):
    cache[key] = value
    while len(cache) > max_size:
        cache.popitem(last=False)

def _stable_hash(data: Any) -> str:
    text = json.dumps(data, ensure_ascii=False, sort_keys=True, default=str)
    return hashlib.md5(text.encode("utf-8")).hexdigest()

```

```

DYNASTY_SCOPE_MAP = {
    "夏": ["夏"], "夏朝": ["夏"], "商": ["商"], "商朝": ["商"], "周": ["西周", "东周"],
    "周朝": ["西周", "东周"], "西周": ["西周"], "东周": ["东周"], "秦": ["秦朝", "秦"], "秦朝": ["秦朝", "秦"],
    "汉": ["汉朝", "西汉", "东汉"], "汉朝": ["汉朝", "西汉", "东汉"], "西汉": ["西汉"], "东汉": ["东汉"],
    "三国": ["三国", "魏国", "蜀国", "吴国"], "晋": ["晋朝", "西晋", "东晋"], "晋朝": ["晋朝", "西晋", "东晋"],
    "西晋": ["西晋"], "东晋": ["东晋"], "隋": ["隋朝", "隋"], "隋朝": ["隋朝", "隋"], "唐": ["唐朝", "唐"],
    "唐朝": ["唐朝", "唐"], "宋": ["宋朝", "北宋", "南宋"], "宋朝": ["宋朝", "北宋", "南宋"], "北宋": ["北宋"],

```



```

        self.relation_rules[relation].append(rule)

print(f"规则索引构建完成: "
      f"{len(self.relation_rules)} 种具体关系, {len(self.composite_rules)} 条复合规则")

def _find_applicable_rules(self,
                           relation: Optional[str] = None) -> List[Dict]: """ 查找适用于给定关系的规则
    Args: relation: 具体关系 Returns: 适用规则列表 """ applicable_rules = []

    # 按具体关系查找
    if relation and relation in self.relation_rules:
        applicable_rules.extend(self.relation_rules[relation])

    # 按优先级排序
    if applicable_rules:
        applicable_rules.sort(key=lambda x: x.get('priority', 0), reverse=True)

    return applicable_rules

def get_entity_relationships(self, entity_id: int, neo4j_db) -> List[Dict]:
    """ 获取实体的关系 Args: entity_id: 实体 ID neo4j_db: Neo4j 数据库实例 Returns: 关系列表 """
    try:
        # 获取出向关系
        query_outgoing = f""" MATCH (n)-[r]->(m) WHERE ID(n) = {entity_id}
        RETURN n, r, m, 'outgoing' as direction """
        # 获取入向关系
        query_incoming = f""" MATCH (n)<-[r]-(m) WHERE ID(n) = {entity_id}
        RETURN n, r, m, 'incoming' as direction """
        # 合并结果
        results_outgoing = neo4j_db.graph.run(query_outgoing).data()
        results_incoming = neo4j_db.graph.run(query_incoming).data()

        relationships = []
        processed_relations = set() # 用于去重

        # 处理所有结果
        for result in results_outgoing + results_incoming:
            source_node = result['n'] if result['direction'] == 'outgoing' else result['m']
            target_node = result['m'] if result['direction'] == 'outgoing' else result['n']
            relation = result['r']

            # 创建关系的唯一标识, 避免重复
            relation_id = f"{source_node.identity}_{target_node.identity}_{type(relation).__name__}"
            if relation_id in processed_relations:
                continue
            processed_relations.add(relation_id)
            rel_type = type(relation).__name__
            rel_info = { 'source': {
                'id': source_node.identity, 'name': source_node.get('name', ''),
                'type': list(source_node.labels)[0] if source_node.labels else '',
                'properties': {k: v for k, v in source_node.items()} }, 'target': {
                'id': target_node.identity, 'name': target_node.get('name', ''),

```

```

        'type': list(target_node.labels)[0] if target_node.labels else '',
        'properties': {k: v for k, v in target_node.items()} },
        'relation': rel_type,
        'properties': {k: v for k, v in relation.items()}},
        'direction': result['direction'] } relationships.append(rel_info)
    print(f"获取到实体 ID({entity_id})的 {len(relationships)} 个关系")
    return relationships
except Exception as e:
    print(f"获取实体关系时出错: {str(e)}")
    return []

```

```

def search_paths_between_entities(self, entity1_id: int, entity2_id: int,

```

```

                                neo4j_db, max_depth: Optional[int] = None) -> List[Dict]:

```

""" 搜索两个实体之间的路径 Args: entity1_id: 第一个实体 ID entity2_id: 第二个实体 ID neo4j_db: Neo4j 数据库实例

max_depth: 最大搜索深度 Returns: 路径列表 """

```

if max_depth is None:

```

```

    max_depth = self.max_depth

```

```

try:

```

```

    # 添加检查: 如果起始和结束节点相同, 则跳过路径搜索

```

```

    if entity1_id == entity2_id:

```

```

        print(f"跳过相同节点的路径搜索: {entity1_id} -> {entity1_id}")

```

```

        return []

```

```

    # 查询从 entity1 到 entity2 的有向路径

```

```

    query_forward = f""" MATCH path = shortestPath((n)-[*1..{max_depth}]->(m))

```

```

    WHERE ID(n) = {entity1_id} AND ID(m) = {entity2_id} RETURN path """

```

```

    # 查询从 entity2 到 entity1 的有向路径

```

```

    query_backward = f""" MATCH path = shortestPath((n)-[*1..{max_depth}]->(m))

```

```

    WHERE ID(n) = {entity2_id} AND ID(m) = {entity1_id} RETURN path """

```

```

    # 合并结果

```

```

    results = []

```

```

    forward_results = neo4j_db.graph.run(query_forward).data()

```

```

    backward_results = neo4j_db.graph.run(query_backward).data()

```

```

    # 如果有正向路径, 优先使用正向路径

```

```

    if forward_results:

```

```

        results = forward_results

```

```

    elif backward_results:

```

```

        results = backward_results

```

```

    paths = []

```

```

    for result in results:

```

```

        path = result.get('path')

```

```

        if not path:

```

```

            continue nodes = list(path.nodes) rels = list(path.relationships)

```

```

            path_data = [] last_node_name = None last_node_id = None

```

```

            for i, node in enumerate(nodes):

```

```

        node_data = { 'name': node['name'], 'id': node.identity,
            'type': list(node.labels)[0] if node.labels else '' }
        # 添加所有节点属性
        for key, value in node.items():
            if key != 'name' and key != 'id' and key != 'type':
                node_data[key] = value

        # 添加关系信息
        if i > 0 and i-1 < len(rels):
            rel = rels[i-1]
            rel_type = type(rel).__name__

            if rel.start_node.identity == last_node_id:
                node_data['relation'] = f"{last_node_name} -{rel_type}-> {node['name']}"
                node_data['relation_direction'] = 'outgoing'
            else:
                node_data['relation'] = f"{last_node_name} <-{rel_type}- {node['name']}"
                node_data['relation_direction'] = 'incoming'

            node_data['relation_type'] = rel_type
            node_data['relation_properties'] = {k: v for k, v in rel.items()}

        last_node_name = node['name'] last_node_id = node.identity
        path_data.append(node_data) paths.append({ 'path': path_data,
            'length': len(rels),
            'start_entity': path_data[0]['name'] if path_data else None,
            'end_entity': path_data[-1]['name'] if path_data else None
        })

    print(f"找到 {len(paths)} 条从 {entity1_id} 到 {entity2_id} 的路径")
    return paths

except Exception as e:
    print(f"搜索路径时出错: {str(e)}")
    return []

def apply_inference_rules(self, relationships: List[Dict]) -> List[Dict]:
    """应用推理规则，生成新的关系"""
    inferred_relationships = []

    # 第一步：应用简单规则（现有逻辑）
    for rel in relationships:
        # 使用原始关系（非格式化）来匹配规则
        relation = rel.get('properties', {}).get('relation_type', '')

        # 跳过已经是推理关系的条目，避免重复推理
        if rel.get('properties', {}).get('inferred', False):
            continue

```

```

# 细分规则
specific_rules = []      # 细分规则

# 获取可应用的规则
for rule in self.rules:
    condition = rule.get('condition', {})
    # 跳过复合规则，复合规则单独处理
    if condition.get('composite', False):
        continue
    rule_relation = condition.get('relation', '')
    if rule_relation:
        if rule_relation == relation:
            specific_rules.append(rule)
            applicable_rules = specific_rules
if not applicable_rules:
    continue

# 应用规则
for rule in applicable_rules:
    # 获取规则的推理部分
    inference = rule.get('inference', {})
    inferred_relation = inference.get('relation', '')
    direction = inference.get('direction', 'forward')
    # 获取规则中定义的原始关系作为 derived_from
    rule_relation = rule.get('condition', {}).get('relation', '')
    if not rule_relation:
        rule_relation = relation

    # 创建新关系
    inferred_rel = { 'source': { 'id': rel['source']['id'],
    'name': rel['source']['name'], 'type': rel['source']['type'] }, 'target': {
    'id': rel['target']['id'], 'name': rel['target']['name'],
    'type': rel['target']['type'] }, 'relation': inferred_relation, 'properties': {
    'inferred': True, 'rule_id': rule['rule_id'],
    'rule_name': rule.get('name', ''), 'derived_from': relation } }
    # 处理其他类型关系或未指定类型关系
    if direction == "reverse":
        inferred_rel['source'], inferred_rel['target'] = inferred_rel['target'],
inferred_rel['source']

    # 在关系对象上添加 inferred 标记，便于前端识别
    inferred_rel['inferred'] = True

    # 记录处理日志
    print(f"推理关系: {inferred_rel['source']['name']} --[{inferred_relation}]-->
{inferred_rel['target']['name']}")
    print(f"  基于原始关系: {rel['source']['name']} --[{relation]}--> {rel['target']['name']}")
    print(f"  规则定义的关系: {rule_relation}")

    inferred_relationships.append(inferred_rel)

```

```

# 第二步：应用复合规则
self._apply_composite_rules(relationships, inferred_relationships)

print(f"根据规则推理出 {len(inferred_relationships)} 个新关系")
return inferred_relationships

def _apply_composite_rules(self, relationships: List[Dict], inferred_relationships: List[Dict]):
    """应用复合规则，处理需要多步关系的推理"""
    # 筛选出复合规则
    composite_rules = [rule for rule in self.rules if rule.get('condition', {}).get('composite', False)]

    if not composite_rules:
        print("没有找到复合规则，跳过复合规则处理")
        return

    print(f"开始处理 {len(composite_rules)} 个复合规则")

    # 构建关系索引，便于快速查找
    # 关系索引结构: {source_id: {target_id: [relation1, relation2, ...]}}
    relation_index = {}

    # 构建双向索引，适用于各种方向的关系查询
    for rel in relationships:
        source_id = rel['source']['id']
        target_id = rel['target']['id']
        relation = rel['relation']
        # 记录关系类型
        rel_info = {'relation': relation, 'rel_obj': rel} # 存储完整的关系对象，便于后续处理
        # 添加到索引，按源节点索引
        if source_id not in relation_index:
            relation_index[source_id] = {}
        if target_id not in relation_index[source_id]:
            relation_index[source_id][target_id] = []
        relation_index[source_id][target_id].append(rel_info)

    # 处理每个复合规则
    for rule in composite_rules:
        condition = rule.get('condition', {})
        path_length = condition.get('path_length', 2) # 默认为2步路径
        relation_type = condition.get('relation', '')

        # 目前仅实现2步路径的规则
        if path_length != 2:
            print(f"暂不支持长度为 {path_length} 的路径规则: {rule['name']}")
            continue

        print(f"处理复合规则: {rule['name']}, 关系类型: {relation_type}, 路径长度: {path_length}")

        # 对于2步路径的规则，寻找形如 A--[rel]-->B--[rel]-->C 的路径
        # 即查找满足的 A→B 和 B→C 关系

```

```

found_paths = []

# 遍历所有可能的第一步关系
for source_id, targets in relation_index.items():
    for middle_id, relations in targets.items():
        # 检查 A→B 关系是否符合规则要求的关系类型
        first_step_relations = [r for r in relations if r['relation'] == relation_type]
        if not first_step_relations:
            continue

        # 检查 B 是否有后续关系
        if middle_id in relation_index:
            # 遍历 B 的所有目标节点
            for target_id, second_relations in relation_index[middle_id].items():
                # 避免自环
                if target_id == source_id:
                    continue

                # 检查 B→C 关系是否符合规则要求的关系类型
                second_step_relations = [r for r in second_relations if r['relation'] ==
relation_type]

                if not second_step_relations:
                    continue

                # 找到一条符合条件的 2 步路径
                first_rel = first_step_relations[0]['rel_obj']
                second_rel = second_step_relations[0]['rel_obj'] path = {
                    'start': { 'id': source_id, 'name': first_rel['source']['name'],
                    'type': first_rel['source']['type'] }, 'middle': { 'id': middle_id,
                    'name': first_rel['target']['name'],
                    'type': first_rel['target']['type'] }, 'end': { 'id': target_id,
                    'name': second_rel['target']['name'],
                    'type': second_rel['target']['type'] },
                    'first_relation': first_rel, 'second_relation': second_rel }
                found_paths.append(path)

# 应用推理规则，根据找到的路径创建新的关系
inference = rule.get('inference', {})
inferred_relation = inference.get('relation', '')
direction = inference.get('direction', 'forward')
for path in found_paths:
    # 创建新的推理关系
    new_rel = { 'source': path['start'], 'target': path['end'],
        'relation': inferred_relation, 'properties': { 'inferred': True,
        'rule_id': rule['rule_id'], 'rule_name': rule.get('name', ''),
        'derived_from': f"{relation_type}链", 'composite': True,
        'path_length': path_length } }
    # 处理关系方向
    if direction == "reverse":
        new_rel['source'], new_rel['target'] = new_rel['target'], new_rel['source']

```

```

        # 标记为推理关系
        new_rel['inferred'] = True

        print(f"复合规则推理: {new_rel['source']['name']} --[{inferred_relation}]-> {new_rel['target']['name']}")
        print(f"  基于路径: {path['start']['name']} --[{relation_type}]-> {path['middle']['name']} --[{relation_type}]-> {path['end']['name']}")
        print(f"  规则: {rule.get('name', '')}")

        # 添加到推理关系集合
        inferred_relationships.append(new_rel)

    print(f"复合规则 {rule['name']} 推理出 {len(found_paths)} 个新关系")

def _build_inference_prompt(self,
                             question: str, entities: List[str], entity_info: List[Dict],
                             relationships: List[Dict], paths: List[Dict]) -> str:
    """构建推理提示词（精简版，减少 token 消耗）"""
    # 预处理用户问题 - 移除多余空格并确保问题以问号结尾
    processed_question = question.strip()
    if processed_question and not processed_question.endswith(('?', ' ? ')):
        processed_question += ' ? '

    # 格式化实体信息为文本（只保留关键属性）
    entity_info_text = ""
    for i, info in enumerate(entity_info[:10]): # 限制最多 10 个实体
        if info:
            properties = info.get('properties', {})
            # 只保留关键属性
            key_props = []
            for k in ['dynasty', 'start_date', 'end_date', 'Place', 'Result', 'Role']:
                if k in properties and properties[k]:
                    key_props.append(f"{k}: {properties[k]}")
            props_text = ", ".join(key_props)
            entity_info_text += f"{i+1}. {info.get('name', '')} ({info.get('type', '')})"
            if props_text:
                entity_info_text += f" - {props_text}"
            entity_info_text += "\n"

    # 分离原始关系和推理关系
    original_relationships = []
    inferred_relationships = []

    for rel in relationships:
        if rel.get('properties', {}).get('inferred', False) or rel.get('inferred', False):
            inferred_relationships.append(rel)
        else:
            original_relationships.append(rel)

```

```

# 格式化关系信息为文本（限制数量）
relationships_text = ""
if original_relationships:
    relationships_text += "【图谱关系】\n"
    for i, rel in enumerate(original_relationships[:15]): # 最多15条
        source = rel.get('source', {}).get('name', '')
        target = rel.get('target', {}).get('name', '')
        relation = rel.get('relation', '')
        relationships_text += f"{source} →[{relation}]→ {target}\n"

if inferred_relationships:
    relationships_text += "\n【推理关系】\n"
    for i, rel in enumerate(inferred_relationships[:10]): # 最多10条
        source = rel.get('source', {}).get('name', '')
        target = rel.get('target', {}).get('name', '')
        relation = rel.get('relation', '')
        relationships_text += f"{source} →[{relation}]→ {target}\n"

# 格式化路径信息为文本（限制数量）
paths_text = ""
for i, path_data in enumerate(paths[:3]): # 最多3条路径
    path = path_data.get('path', [])
    if path:
        path_str = path[0].get('name', '')
        for j in range(1, len(path)):
            node = path[j]
            relation = node.get('relation_type', '')
            path_str += f" →[{relation}]→ {node.get('name', '')}"
        paths_text += f"{i+1}. {path_str}\n"

# 构建精简版提示词
prompt = f"""基于知识图谱回答历史战争问题。 【问题】 {processed_question} 【相关实体】 {', '.join(entities[:8])}
【实体信息】 {entity_info_text if entity_info_text else '无'} 【实体关系】
{relationships_text if relationships_text else '无'} 【关系路径】
{paths_text if paths_text else '无'} 【要求】 1. 直接回答问题，不要说“根据信息”等引导语 2. 区分图谱
事实和推理关系
3. 使用中文回答 4. 枚举问题需覆盖全部匹配事件 """
return prompt

def _build_dynasty_event_list_prompt(self,
                                     question: str, dynasty_scope: Dict[str, Any],
                                     event_info: List[Dict]) -> str:
    """为朝代战争枚举问题构建结构化提示词，减少遗漏和跑题。"""
    processed_question = question.strip()
    if processed_question and not processed_question.endswith(('?', ' ')):
        processed_question += ' ? '

    event_lines = []
    for index, info in enumerate(event_info, 1):

```

```

        props = info.get('properties', {})
        def field(label, value):
            value_text = "" if value in (None, "None") else str(value).strip()
            return f"{label}={value_text}" if value_text else ""

        fields = [ field("事件名", info.get('name', '')),
                    field("朝代", props.get('dynasty', '')), field("时间", props.get('start_date', '')),
                    field("地点", props.get('Place', '')), field("发起方", props.get('aggressor', '')),
                    field("防御方", props.get('defender', '')), field("结果", props.get('Result', '')),
                    field("影响", props.get('Impact', '')), ]
        event_lines.append(f"{index}. " + "; ".join([item for item in fields if item]))

    return f"""

## 用户原始问题
{processed_question}

## 问题类型
朝代战争事件枚举

## 检索范围
用户询问的是"{dynasty_scope.get('display', '')}"相关战争。
后端已将图谱范围限定为朝代属于：{' , ' .join(dynasty_scope.get('dynasties', []))}

## 图谱事件清单
共 {len(event_info)} 条事件，必须全部覆盖：
{chr(10).join(event_lines) if event_lines else '无'}

## 回答要求
1. 必须逐条列出上方"图谱事件清单"中的全部 {len(event_info)} 条事件，不要只选代表性事件。
2. 不要添加清单之外的事件，不要把其他朝代事件混入答案。 3. 每条事件优先包含：事件名、时间、结果；如果字段为空或"不详"，可以省略该字段。
4. 可以按朝代小标题分组，但不能把多个事件合并成一句。 5. 不要给出"最准确答案是某一事件"这类单选结论。 6. 最后可以用一句话说明"以上为图谱中检索到的完整清单"。 """

    def _format_event_record_lines(self, event_info: List[Dict]) -> str:
        """把事件节点整理成结构化清单，供专用问答提示词使用。"""
        event_lines = []
        for index, info in enumerate(event_info, 1):
            props = info.get('properties', {})

            def field(label, value):
                value_text = "" if value in (None, "None") else str(value).strip()
                return f"{label}={value_text}" if value_text else ""

            fields = [ field("事件名", info.get('name', '')),
                        field("朝代", props.get('dynasty', '')), field("时间", props.get('start_date', '')),
                        field("地点", props.get('Place', '')), field("发起方", props.get('aggressor', '')),
                        field("防御方", props.get('defender', '')), field("相关人物", props.get('person', '')),
                        field("参与对象", props.get('matched_participant', '')),
                        field("参与关系", props.get('participant_relation', '')),

```

```

        field("结果", props.get('Result', '')), field("影响", props.get('Impact', '')),
        field("备注", props.get('Remark', '')), ]
        event_lines.append(f"{index}. " + "; ".join([item for item in fields if item]))
    return "\n".join(event_lines)

```

```

def _build_event_detail_prompt(self, question: str, event_info: List[Dict]) -> str:

```

```

    """为事件详情问题构建结构化提示词。"""

```

```

    processed_question = question.strip()

```

```

    if processed_question and not processed_question.endswith(('?', ' ? ')):

```

```

        processed_question += ' ? '

```

```

    return f"""

```

```

## 用户原始问题

```

```

{processed_question}

```

```

## 问题类型

```

```

战争事件详情

```

```

## 图谱事件记录

```

```

{self._format_event_record_lines(event_info) if event_info else '无'}

```

```

## 回答要求

```

```

1. 只基于上方图谱事件记录回答，不要添加图谱外史实。 2. 根据用户问的字段重点回答；如果用户没有限定字段，按“
时间、地点、参战方/人物、结果、影响”组织。
3. 如果某字段为空或“不详”，明确说明图谱未提供该字段。 4. 如果匹配到多个事件，先说明匹配到多个，再分别回答。
"""

```

```

def _build_participant_event_prompt(self, question: str, event_info: List[Dict]) -> str:

```

```

    """为人物/组织参与事件问题构建结构化提示词。"""

```

```

    processed_question = question.strip()

```

```

    if processed_question and not processed_question.endswith(('?', ' ? ')):

```

```

        processed_question += ' ? '

```

```

    return f"""

```

```

## 用户原始问题

```

```

{processed_question}

```

```

## 问题类型

```

```

人物或组织参与战争枚举

```

```

## 图谱事件清单

```

```

共 {len(event_info)} 条事件：

```

```

{self._format_event_record_lines(event_info) if event_info else '无'}

```

```

## 回答要求

```

```

1. 必须覆盖上方清单中的全部事件，不要只选代表性事件。 2. 每条事件说明事件名、参与关系、时间和结果；字段为空
或“不详”时可以省略。 3. 不要添加清单之外的事件。
4. 最后说明这些记录来自当前图谱的直接关系。 """

```

```

def generate_response_with_llm(self,

```

```

    question: str, entities: List[str], entity_info: List[Dict],

```

```

        relationships: List[Dict], paths: List[Dict],
        question_type: str = "general",
        metadata: Optional[Dict[str, Any]] = None) -> str: """
        使用大模型生成回答 Args: question: 用户问题 entities: 实体列表
        entity_info: 实体信息列表 relationships: 关系列表 paths: 路径列表 Returns:
        生成的回答 """ metadata = metadata or {}

    if question_type == "dynasty_event_list":
        user_prompt = self._build_dynasty_event_list_prompt( question=question,
            dynasty_scope=metadata.get("dynasty_scope", {}), event_info=entity_info )
    elif question_type == "event_detail":
        user_prompt = self._build_event_detail_prompt(question, entity_info)
    elif question_type == "participant_event_list":
        user_prompt = self._build_participant_event_prompt(question, entity_info)
    else:
        user_prompt = self._build_inference_prompt( question=question, entities=entities,
            entity_info=entity_info, relationships=relationships, paths=paths )
        answer_cache_key = ( "answer", self.model_name, question_type, _stable_hash({
            "question": question, "entities": entities, "entity_info": entity_info,
            "relationships": relationships, "paths": paths, "metadata": metadata,
            "prompt": user_prompt, }) )
        cached_answer = _lru_get(self._answer_cache, answer_cache_key)
        if cached_answer is not None:
            print(f"命中问答生成缓存: {question_type} / {question}")
            return cached_answer

    try:
        # 记录模型调用开始时间
        start_time = time.time()
        print(f"调用 {self.model_name} 模型生成回答...")

        # 增强的 system 提示词, 强调对用户问题的准确理解
        if question_type in {"dynasty_event_list", "participant_event_list"}:
            system_prompt = """你是一位中国历史战争知识图谱问答助手。当前任务是根据后端提供的结构化事件
            清单回答枚举类问题。
            必须完整覆盖清单中的每一条事件; 不得自行删减、合并、补充清单外事件; 不得把枚举题回答成单个"最准确答案"。"""
        elif question_type == "event_detail":
            system_prompt = """你是一位中国历史战争知识图谱问答助手。当前任务是根据后端提供的结构化事件
            记录回答事件详情问题。
            只能使用记录中的字段作答; 字段缺失时说明图谱未提供; 不得补充图谱外史实。"""
        else:
            system_prompt = """你是一位专业的中国历史战争研究与事件关系分析专家, 精通中国历代战争事件、
            参战势力、战役过程及其历史背景。
            你的职责是: 1. 只回答用户提出的原始问题, 不要重复或引用我给你的指令内容 2. 不要在回答中说"根据提供的信息"
            或"基于图谱数据"等引导语
            3. 所有回答必须基于知识图谱提供的事实, 不要编造不存在的关系 4. 答案应直接、简洁, 不要添加不必要的解释
            重要: 不要将指令或提示词本身视为用户问题的一部分。用户的原始问题已在提示词开头明确标出。 """

        # 调用大模型生成回答
        response = ollama.chat( model=self.model_name, messages=[ { "role": "system",
            "content": system_prompt }, { "role": "user", "content": user_prompt } ],

```

```

        stream=False, keep_alive="10m", options={"temperature": 0.1} # 添加温度参数 )
        # 记录模型调用耗时
        process_time = time.time() - start_time
        answer = response['message']['content']
        _lru_set(self._answer_cache, answer_cache_key, answer)
        print(f"模型响应成功, 耗时: {process_time:.2f}秒")
        return answer

except Exception as e:
    print(f"模型调用失败: {str(e)}")
    return f"抱歉, 在处理您的问题遇到了技术问题。错误信息: {str(e)}"

def _filter_relevant_entities(self, entities: List[str], question: str, neo4j_db) -> List[str]:
    """ 过滤与用户问题最相关的实体 策略: 1. 首先检查实体是否在知识图谱中存在 2. 优先选择名称较长的实体
    (通常更具体) 3. 限制实体数量, 避免过多无关查询
    4. 检查实体与问题关键词的匹配度 """
    if not entities:
        return []

    print(f"过滤前实体列表: {entities}")

    # 去除问题中的常见停用词和标点
    import re
    # 提取问题中的关键词 (长度>=2 的中文字符序列)
    question_keywords = set(re.findall(r'[\u4e00-\u9fa5]{2,}', question))

    scored_entities = []
    existing_entities = set()
    try:
        query = """
        MATCH (n)
        WHERE any(entity IN $entities WHERE n.name = entity OR n.name CONTAINS entity)
        RETURN DISTINCT n.name as name
        LIMIT 50
        """
        rows = neo4j_db.graph.run(query, entities=entities).data()
        existing_names = [row.get('name', '') for row in rows]
        for entity in entities:
            if any(name == entity or entity in name for name in existing_names):
                existing_entities.add(entity)
    except Exception as e:
        print(f"批量查询实体时出错: {e}")

    for entity in entities:
        score = 0

        if entity in existing_entities:
            score += 10
            score += len(entity) * 0.5

```

```

        for keyword in question_keywords:
            if entity in keyword or keyword in entity:
                score += 5
                break

        if entity in question:
            score += 10
            scored_entities.append((entity, score))
# 按分数降序排序
scored_entities.sort(key=lambda x: x[1], reverse=True)
print(f"实体评分结果: {scored_entities}")

# 只保留分数最高的前4个实体（避免过多无关实体）
max_entities = 4
filtered_entities = [e[0] for e in scored_entities[:max_entities] if e[1] > 0]

# 确保至少有1个实体被返回（如果有的话）
if not filtered_entities and scored_entities:
    filtered_entities = [scored_entities[0][0]]

print(f"过滤后实体列表: {filtered_entities}")
return filtered_entities

def _extract_query_entities(self, question: str, extracted_entities: List[str]) -> List[str]:
    """只返回用户原句中明确出现的实体，用于前端展示。"""
    query_entities = []

    for dynasty in sorted(DYNASTY_SCOPE_MAP.keys(), key=len, reverse=True):
        if dynasty in question and dynasty not in query_entities:
            query_entities.append(dynasty)
            break

    for entity in extracted_entities:
        if entity in question and entity not in query_entities:
            query_entities.append(entity)

    if not query_entities:
        for entity in extracted_entities:
            if entity not in query_entities:
                query_entities.append(entity)
            if len(query_entities) >= 3:
                break

    return query_entities

def _detect_dynasty_event_scope(self, question: str) -> Optional[Dict[str, Any]]:
    """识别“某朝有哪些战争/战役”这类朝代范围查询。"""
    if not any(word in question for word in ["战争", "战役", "战事", "之战", "起义", "叛乱"]):
        return None

```

```

        if not any(word in question for word in ["哪些", "那些", "有哪些", "有那些", "有什么", "列出", "包括", "多少", "几场"]):
            return None

    for dynasty in sorted(DYNASTY_SCOPE_MAP.keys(), key=len, reverse=True):
        if dynasty in question:
            return {
                "display": dynasty, "dynasties": DYNASTY_SCOPE_MAP[dynasty], }
    return None

def _looks_like_event_detail_query(self, question: str, entities: List[str]) -> bool:
    """识别某个战争事件的时间、地点、结果、影响等详情问题。"""
    if not any(entity in question and any(suffix in entity for suffix in ["之战", "战役", "起义", "叛乱"]) for entity in entities):
        return False
    return any(word in question for word in [
        "时间", "何时", "什么时候", "地点", "哪里", "结果", "影响", "原因", "经过",
        "详情", "介绍", "发生", "发起方", "防御方", "参与", "参战"
    ])

def _looks_like_participant_event_query(self, question: str) -> bool:
    """识别人/组织参与了哪些战争。"""
    return any(word in question for word in ["参加", "参与", "参战", "发动", "发起", "指挥", "统帅", "有哪些战争", "有那些战争"]) and any(
        word in question for word in ["战争", "战役", "之战", "事件"]
    )

def _get_dynasty_event_infos(self, neo4j_db, dynasty_values: List[str], limit: int = 40) -> List[Dict]:
    """按朝代属性直接取事件节点，作为模型回答上下文。"""
    cache_key = ("dynasty_events", tuple(dynasty_values), limit)
    cached = _lru_get(self._retrieval_cache, cache_key)
    if cached is not None:
        print(f"命中朝代事件检索缓存: {dynasty_values}")
        return cached

    query = """ MATCH (n:Event) WHERE n.dynasty IN $dynasties RETURN n
    ORDER BY coalesce(n.start_date, ''), n.name LIMIT $limit """
    try:
        rows = neo4j_db.graph.run(query, dynasties=dynasty_values, limit=limit).data()
    except Exception as e:
        print(f"按朝代查询事件失败: {e}")
        return []

    event_infos = []
    seen_ids = set()
    for row in rows:
        node = row.get("n")
        if not node or node.identity in seen_ids:
            continue
        seen_ids.add(node.identity)
        event_infos.append({ "id": node.identity,

```

```

        "name": node.get("name", ""),
        "type": list(node.labels)[0] if node.labels else "Event",
        "properties": {k: v for k, v in node.items()}
    })

```

```

_lru_set(self._retrieval_cache, cache_key, event_infos)
return event_infos

```

```

def _get_event_detail_infos(self, neo4j_db, event_names: List[str], limit: int = 8) -> List[Dict]:
    """按事件名取事件详情。"""

```

```

    if not event_names:
        return []

```

```

    cache_key = ("event_detail", tuple(event_names), limit)
    cached = _lru_get(self._retrieval_cache, cache_key)
    if cached is not None:
        print(f"命中事件详情检索缓存: {event_names}")
        return cached

```

```

    query = """
MATCH (n:Event)
WHERE any(name IN $names WHERE n.name = name OR n.name CONTAINS name OR name CONTAINS n.name)
RETURN n
LIMIT $limit
"""

```

```

    try:
        rows = neo4j_db.graph.run(query, names=event_names, limit=limit).data()
    except Exception as e:
        print(f"查询事件详情失败: {e}")
        return []

```

```

    event_infos = []
    seen_ids = set()
    for row in rows:
        node = row.get("n")
        if not node or node.identity in seen_ids:
            continue
        seen_ids.add(node.identity)
        event_infos.append({
            "id": node.identity,
            "name": node.get("name", ""),
            "type": list(node.labels)[0] if node.labels else "Event",
            "properties": {k: v for k, v in node.items()}
        })

```

```

_lru_set(self._retrieval_cache, cache_key, event_infos)
return event_infos

```

```

def _get_participant_event_infos(self, neo4j_db, participant_names: List[str], limit: int = 40) ->
List[Dict]:

```

```

    """按人物或组织名称取其直接参与的事件。"""
    if not participant_names:

```

```

        return []

    cache_key = ("participant_events", tuple(participant_names), limit)
    cached = _lru_get(self._retrieval_cache, cache_key)
    if cached is not None:
        print(f"命中参与事件检索缓存: {participant_names}")
        return cached

    query = """
    MATCH (p)-[r]-(e:Event)
    WHERE any(name IN $names WHERE p.name = name OR p.name CONTAINS name OR name CONTAINS p.name)
    RETURN DISTINCT e, p.name as participant, type(r) as relation
    ORDER BY coalesce(e.start_date, ''), e.name
    LIMIT $limit
    """

    try:
        rows = neo4j_db.graph.run(query, names=participant_names, limit=limit).data()
    except Exception as e:
        print(f"查询参与事件失败: {e}")
        return []

```

```

    event_infos = []
    seen_ids = set()
    for row in rows:
        node = row.get("e")
        if not node or node.identity in seen_ids:
            continue
        seen_ids.add(node.identity)
        props = {k: v for k, v in node.items()}
        props["matched_participant"] = row.get("participant", "")
        props["participant_relation"] = row.get("relation", "")
        event_infos.append({
            "id": node.identity,
            "name": node.get("name", ""),
            "type": list(node.labels)[0] if node.labels else "Event",
            "properties": props
        })

```

```

    _lru_set(self._retrieval_cache, cache_key, event_infos)
    return event_infos

```

```

def process_question(self, question: str, entity_extractor, neo4j_db) -> Dict:

```

""" 处理用户问题，返回推理结果 Args: question: 用户问题 entity_extractor: 实体提取器实例 neo4j_db:

Neo4j 数据库实例

Returns: 包含回答和可视化数据的字典 """ start_time = time.time()

```

try:
    # 预处理用户问题
    if not question or not question.strip():
        return {

```

```

        'answer': "请提供有效的问题。", 'entities': [],
        'kg_data': {'nodes': [], 'lines': []},
        'process_time': time.time() - start_time }

# 规范化问题格式
processed_question = question.strip()
# 检查问题是否过短或可能无效
if len(processed_question) < 3:
    print(f"警告: 问题过短或可能无效: '{processed_question}'")

print(f"开始处理用户问题: '{processed_question}'")

# 1. 从问题中提取实体
dynasty_scope = self._detect_dynasty_event_scope(processed_question)
if dynasty_scope:
    extracted_entities = [] query_entities = [dynasty_scope["display"]]
    event_detail_query = False participant_event_query = False
else:
    extracted_entities = entity_extractor.extract_entities(processed_question)
    query_entities = self._extract_query_entities(processed_question, extracted_entities)
    event_detail_query = self._looks_like_event_detail_query(processed_question,
extracted_entities)
    participant_event_query = self._looks_like_participant_event_query(processed_question)

# 1.5 过滤实体，只保留最相关的。朝代范围题已由规则明确实体，避免再走一次模型抽取。
if dynasty_scope:
    entities = query_entities.copy()
    print(f"命中朝代事件范围查询，跳过实体抽取模型: {query_entities}")
else:
    entities = self._filter_relevant_entities(extracted_entities, processed_question, neo4j_db)

# 如果没有识别到实体，但问题中可能包含地名相关词汇，尝试进行模糊匹配
if not entities:
    # 检查问题中是否包含可能的地名关键词
    all_keywords = ["之战", "战役", "战事", "战争", "起义", "叛乱", "征讨", "围城", "帝", "王",
"将军", "太守", "丞相", "元帅", "侯", "公", "国", "朝", "军", "部", "盟", "部族", "王朝", "政权", "城", "关",
"州", "郡", "县", "府", "道", "路", "山", "河"]
    has_keyword = any(keyword in processed_question for keyword in all_keywords)

    if has_keyword:
        print(f"未识别到明确实体，但问题中包含战争图谱相关关键词，尝试进行模糊匹配")
        # 提取问题中的所有可能实体词（简单处理，实际应使用 NLP 工具）
        potential_entities = []
        for keyword in all_keywords:
            if keyword in processed_question and len(keyword) > 1: # 避免单字匹配
                potential_entities.append(keyword)

        if potential_entities:
            print(f"从问题中提取潜在地名关键词: {potential_entities}")
            entities = potential_entities

```

```

    if not entities:
        return {
            'answer': '抱歉，我无法从您的问题中识别出任何明确的战争事件、人物、组织或地点实体。请尝试提供更具体的名称，例如“赤壁之战”、“曹操”、“魏国”、“襄阳城”等。',
            'entities': [], 'kg_data': {'nodes': [], 'lines': []},
            'process_time': time.time() - start_time }
    print(f"从问题中识别到的实体: {entities}")
    print(f"用户原句明确实体: {query_entities}")

    # 记录查询涉及的所有节点和关系
    all_entity_info = [] all_relationships = [] all_paths = []
    # 2. 从知识图谱中查询实体信息
    entity_info_map = {} # 实体 ID 到实体信息的映射
    if dynasty_scope:
        all_entity_info = self._get_dynasty_event_infos(neo4j_db, dynasty_scope["dynasties"])
        for info in all_entity_info:
            entity_info_map[info['id']] = info
        query_entities = query_entities or [dynasty_scope["display"]]
        entities = query_entities
    elif event_detail_query:
        event_names = [entity for entity in entities if any(suffix in entity for suffix in ["之战", "战役", "起义", "叛乱"])]
        all_entity_info = self._get_event_detail_infos(neo4j_db, event_names or entities)
        for info in all_entity_info:
            entity_info_map[info['id']] = info
        query_entities = query_entities or event_names or entities
        entities = query_entities
    elif participant_event_query:
        participant_names = query_entities or entities
        all_entity_info = self._get_participant_event_infos(neo4j_db, participant_names)
        for info in all_entity_info:
            entity_info_map[info['id']] = info
        entities = query_entities or entities

    # 并行查询实体信息
    query_entities_list = [] if (dynasty_scope or event_detail_query or participant_event_query)
else entities
    if query_entities_list:
        print(f"开始并行查询 {len(query_entities_list)} 个实体信息...")
        query_start = time.time()

        def query_single_entity(entity):
            """查询单个实体信息"""
            try:
                # 先进行精确查询
                query = f""" MATCH (n) WHERE n.name = '{entity}' RETURN n LIMIT 1 """
                results = neo4j_db.graph.run(query).data()
                if results:

```

```

        node = results[0]['n']
        return [{
            'id': node.identity, 'name': node['name'],
            'type': list(node.labels)[0] if node.labels else '',
            'properties': {k: v for k, v in node.items()} }]

# 尝试模糊查询
fuzzy_query = f"""
MATCH (n)
WHERE n.name CONTAINS '{entity}' OR '{entity}' CONTAINS n.name
RETURN n LIMIT 5 """
fuzzy_results = neo4j_db.graph.run(fuzzy_query).data()
if fuzzy_results:
    return [{
        'id': result['n'].identity,
        'name': result['n']['name'],
        'type': list(result['n'].labels)[0] if result['n'].labels else '',
        'properties': {k: v for k, v in result['n'].items()}
    } for result in fuzzy_results]

return []

except Exception as e:
    print(f"查询实体 '{entity}' 失败: {e}")
    return []

```

使用线程池并行查询

```

with ThreadPoolExecutor(max_workers=min(4, len(query_entities_list))) as executor:
    future_to_entity = {
        executor.submit(query_single_entity, entity): entity
        for entity in query_entities_list
    }

```

```

for future in as_completed(future_to_entity):
    entity = future_to_entity[future]
    try:
        entity_infos = future.result()
        for info in entity_infos:
            all_entity_info.append(info)
            entity_info_map[info['id']] = info
    except Exception as e:
        print(f"处理实体 '{entity}' 结果失败: {e}")

```

```

query_time = time.time() - query_start
print(f"实体信息查询完成, 耗时: {query_time:.2f}秒, 找到 {len(all_entity_info)} 个实体")

```

如果没有找到任何实体信息, 给出更友好的回复

```

if not all_entity_info:

```

```

    return {
        'answer': f"抱歉, 我无法在知识库中找到与'{', '.join(entities)}'相关的实体信息。请尝试其
他名称或检查拼写是否正确。",
    }

```

```

        'entities': entities, 'kg_data': {'nodes': [], 'lines': []},
        'process_time': time.time() - start_time }

# 3. 并行获取实体关系
print(f"开始并行查询 {len(all_entity_info)} 个实体的关系...")
relation_start = time.time()

def query_entity_relations(info):
    """查询单个实体的关系"""
    try:
        return self.get_entity_relationships(info['id'], neo4j_db)
    except Exception as e:
        print(f"查询实体 '{info['name']}' 关系失败: {e}")
        return []

with ThreadPoolExecutor(max_workers=min(4, len(all_entity_info))) as executor:
    future_to_info = {
        executor.submit(query_entity_relations, info): info
        for info in all_entity_info
    }

    for future in as_completed(future_to_info):
        info = future_to_info[future]
        try:
            relationships = future.result()
            all_relationships.extend(relationships)
        except Exception as e:
            print(f"处理实体 '{info['name']}' 关系结果失败: {e}")

relation_time = time.time() - relation_start
print(f"实体关系查询完成, 耗时: {relation_time:.2f} 秒, 找到 {len(all_relationships)} 条关系")

# 4. 并行搜索实体之间的路径
if len(all_entity_info) >= 2 and not (dynasty_scope or event_detail_query or
participant_event_query):
    print(f"开始并行搜索实体间路径...")
    path_start = time.time()

    # 构建实体对列表
    entity_pairs = []
    processed_entity_pairs = set()

    for i in range(len(all_entity_info)):
        for j in range(i+1, len(all_entity_info)):
            entity1_id = all_entity_info[i]['id']
            entity2_id = all_entity_info[j]['id']

            # 跳过相同的实体 ID
            if entity1_id == entity2_id:
                continue

```

```

        # 创建一个排序后的实体 ID 对作为键，确保不重复查询
        entity_pair = tuple(sorted([entity1_id, entity2_id]))
        if entity_pair in processed_entity_pairs:
            continue
        processed_entity_pairs.add(entity_pair)
        entity_pairs.append((entity1_id, entity2_id))

def search_path_for_pair(pair):
    """搜索一对实体之间的路径"""
    entity1_id, entity2_id = pair
    try:
        return self.search_paths_between_entities(entity1_id, entity2_id, neo4j_db)
    except Exception as e:
        print(f"搜索路径 {entity1_id} -> {entity2_id} 失败: {e}")
        return []

# 使用线程池并行搜索路径
if entity_pairs:
    with ThreadPoolExecutor(max_workers=min(4, len(entity_pairs))) as executor:
        future_to_pair = {
            executor.submit(search_path_for_pair, pair): pair
            for pair in entity_pairs
        }

        for future in as_completed(future_to_pair):
            pair = future_to_pair[future]
            try:
                paths = future.result()
                all_paths.extend(paths)
            except Exception as e:
                print(f"处理路径 {pair} 结果失败: {e}")

    path_time = time.time() - path_start
    print(f"路径搜索完成，耗时: {path_time:.2f} 秒，找到 {len(all_paths)} 条路径")

# 5. 应用推理规则
inferred_relationships = self.apply_inference_rules(all_relationships)

# 原始关系与推理关系分开处理
original_relationships = all_relationships.copy()
all_relationships.extend(inferred_relationships)

# 6. 使用大模型生成回答
answer_entities = entities
question_type = "general"
answer_metadata = {}
if dynasty_scope:
    question_type = "dynasty_event_list"
    answer_metadata["dynasty_scope"] = dynasty_scope
    answer_entities = [

```

```

        f"{dynasty_scope['display']}] (仅限朝代属于 {', '.join(dynasty_scope['dynasties'])} 的战争事件)"
    ]
elif event_detail_query:
    question_type = "event_detail"
elif participant_event_query:
    question_type = "participant_event_list"
    answer = self.generate_response_with_llm(
        question=processed_question, # 使用处理后的问题 entities=answer_entities,
        entity_info=all_entity_info, relationships=all_relationships, paths=all_paths,
        question_type=question_type, metadata=answer_metadata )
# 7. 构建知识图谱可视化数据 (只使用原始关系, 不含推理关系)
kg_data = self._convert_to_visual_data(all_entity_info, original_relationships, all_paths)

# 8. 格式化关系数据为三元组格式 (包含原始和推理关系, 但推理关系明确标注)
context = self._format_relations_for_context(original_relationships, inferred_relationships)

process_time = time.time() - start_time
print(f"问题处理完成, 总耗时: {process_time:.2f}秒")

return {
    'answer': answer, 'entities': entities,
    'query_entities': query_entities or entities, 'kg_data': kg_data,
    'context': context, # 添加格式化后的关系上下文 'process_time': process_time }
except Exception as e:
    print(f"处理问题时出错: {str(e)}")
    import traceback
    traceback.print_exc()

return {
    'answer': f"抱歉, 处理您的问题遇到了技术问题: {str(e)}",
    'entities': entities if 'entities' in locals() else [],
    'kg_data': {'nodes': [], 'lines': []}, 'error': str(e),
    'process_time': time.time() - start_time }
def _convert_to_visual_data(self, entities, relationships, paths):
    """将实体、关系和路径转换为可视化数据格式""" nodes = [] lines = [] node_ids = set() line_pairs =
set()
def _clean_node_properties(properties):
    cleaned = {}
    for key, value in (properties or {}).items():
        if key in {'id', 'name', 'type'}:
            continue
        if value in (None, ''):
            continue
        if isinstance(value, (str, int, float, bool)):
            cleaned[key] = value
    return cleaned

def _upsert_node(node_id, name, node_type, properties=None):

```

```

cleaned_properties = _clean_node_properties(properties)

if node_id not in node_ids:
    node_data = { 'id': node_id, 'name': name, 'type': node_type }
    node_data.update(cleaned_properties) nodes.append(node_data)
    node_ids.add(node_id) return
for node in nodes:
    if node.get('id') != node_id:
        continue
    if name and not node.get('name'):
        node['name'] = name
    if node_type and not node.get('type'):
        node['type'] = node_type
    for key, value in cleaned_properties.items():
        if not node.get(key):
            node[key] = value
    return

for entity in entities:
    _upsert_node( entity['id'], entity.get('name', ''), entity.get('type', ''),
    entity.get('properties', {}) )
for path_data in paths:
    path = path_data.get('path', [])
    last_node = None

    for i, node in enumerate(path):
        node_id = node.get('id') _upsert_node( node_id, node.get('name', '未命名'),
        node.get('type', ''), { k: v for k, v in node.items()
            if k not in {'id', 'name', 'type', 'relation', 'relation_direction',
'relation_type', 'relation_properties'}
        }
        )

        if i > 0 and last_node:
            relation_type = node.get('relation_type', '')
            from_id = last_node.get('id')
            to_id = node_id
            relation_key = (min(from_id, to_id), max(from_id, to_id), relation_type)

            if relation_key not in line_pairs:
                line_data = { 'from': from_id, 'to': to_id, 'text': relation_type,
                    'relation_direction': node.get('relation_direction', 'outgoing')
                } lines.append(line_data) line_pairs.add(relation_key) last_node = node
            relation_ids = set()

for rel in relationships:
    from_id = rel['source']['id'] to_id = rel['target']['id']
    relation_type = rel['relation'] _upsert_node( from_id, rel['source']['name'],
    rel['source']['type'], rel['source'].get('properties', {}) ) _upsert_node( to_id,
    rel['target']['name'], rel['target']['type'], rel['target'].get('properties', {}) )

```

```

relation_id = f"{from_id}_{to_id}_{relation_type}"
if relation_id not in relation_ids:
    relation_ids.add(relation_id) line_data = { 'from': from_id, 'to': to_id,
    'text': relation_type }
    if 'properties' in rel:
        for k, v in rel['properties'].items():
            line_data[k] = v

    if 'direction' in rel:
        line_data['direction'] = rel['direction']

    if rel.get('properties', {}).get('inferred', False):
        line_data['inferred'] = True
    elif rel.get('inferred', False):
        line_data['inferred'] = True lines.append(line_data) unique_lines = []
        processed_pairs = set()
for line in lines:
    from_id = line['from'] to_id = line['to'] relation_type = line['text']
    entity_pair = tuple(sorted([from_id, to_id]))
    relation_key = (*entity_pair, relation_type)
    if relation_key in processed_pairs and not line.get('inferred', False):
        continue processed_pairs.add(relation_key) unique_lines.append(line)
return {
    'nodes': nodes, 'lines': unique_lines }

def _format_relations_for_context(self, relations: List[Dict[str, Any]], inferred_relations:
List[Dict[str, Any]]) -> str:
    """将关系信息格式化为易读的上下文信息"""
    text_parts = []

    # 添加简洁的标题
    text_parts.append("📖 知识图谱参考信息")
    text_parts.append("")

    # 收集所有实体信息
    all_entities = set()
    entity_info_map = {}

    def collect_entity_info(rel_list):
        for rel in rel_list:
            source = rel.get("source", {})
            target = rel.get("target", {})

            for entity in [source, target]:
                name = entity.get("name", "")
                entity_type = entity.get("type", "")
                if name and name not in entity_info_map:
                    entity_info_map[name] = entity_type all_entities.add(name)
            collect_entity_info(relations) collect_entity_info(inferred_relations)

    # 输出实体列表

```

```

if all_entities:
    text_parts.append("【涉及实体】")
    for name in sorted(all_entities):
        entity_type = entity_info_map.get(name, "")
        type_icon = {"Event": "📅", "Person": "👤", "Organization": "🏢", "Place":
"📍"}.get(entity_type, "•")
        text_parts.append(f" {type_icon} {name} ({entity_type})")
    text_parts.append("")

# 输出直接关系
if relations:
    # 过滤掉可能混入的推理关系
    direct_relations = [r for r in relations if not r.get("properties", {}).get("inferred", False)]

    if direct_relations:
        text_parts.append("【直接关系】(来自知识图谱)")
        text_parts.append("")

        for i, relation in enumerate(direct_relations[:15], 1): # 限制数量避免过长
            source = relation.get("source", {}).get("name", "未知")
            target = relation.get("target", {}).get("name", "未知")
            relation_type = relation.get("relation", "相关")
            # 简化关系描述
            text_parts.append(f"{i}. {source} → 【{relation_type}】 → {target}")

        if len(direct_relations) > 15:
            text_parts.append(f"... 还有 {len(direct_relations) - 15} 条关系")

        text_parts.append("")

# 输出推理关系
if inferred_relations:
    text_parts.append("【推理关系】(基于规则推导)")
    text_parts.append("")

    for i, relation in enumerate(inferred_relations[:10], 1): # 限制数量
        source = relation.get("source", {}).get("name", "未知")
        target = relation.get("target", {}).get("name", "未知")
        relation_type = relation.get("relation", "相关")
        derived_from = relation.get("properties", {}).get("derived_from", "")

        # 简化显示
        text_parts.append(f"{i}. {source} → 【{relation_type}】 → {target}")
        if derived_from:
            text_parts.append(f"    (基于: {derived_from})")

    if len(inferred_relations) > 10:
        text_parts.append(f"... 还有 {len(inferred_relations) - 10} 条推理关系")

```

```

        text_parts.append("")

    if not relations and not inferred_relations:
        return "未找到相关实体关系信息"

    return "\n".join(text_parts)

```

backend/entity_extract/extractor.py

说明：实体提取器

"""实体提取器模块 功能：使用大模型从文本中提取实体 - extract_entities(text)：提取战争事件、人物、组织、地点
- _build_prompt(text)：构建提取提示词 - _parse_entities_from_response()：解析模型响应
模型：deepseek-r1:7b（通过 ollama 本地服务）"""

```

import os
import json
import re
import ollama
import time
from collections import OrderedDict

```

```

def _cache_get(cache, key):
    if key not in cache:
        return None
    value = cache.pop(key)
    cache[key] = value
    return list(value)

```

```

def _cache_set(cache, key, value, max_size=128):
    cache[key] = list(value)
    while len(cache) > max_size:
        cache.popitem(last=False)

```

```

class Extractor:
    """ 基于大模型的地名实体提取器 使用 Ollama 提供的大模型能力和 deepseek-r1 模型提取所有地名实体 """
    # 常见战争事件后缀
    WAR_EVENT_SUFFIXES = ['之战', '战役', '起义', '叛乱', '围城', '会战', '大战', '征讨', '远征']

    # 常见历史人物称号
    PERSON_TITLES = ['帝', '王', '将军', '太守', '丞相', '元帅', '侯', '公', '君', '主', '帅', '将']

    # 常见组织/政权关键词
    ORG_KEYWORDS = ['国', '朝', '军', '部', '盟', '部族', '王朝', '政权', '军队', '势力']

    # 常见地点关键词
    PLACE_KEYWORDS = ['城', '关', '州', '郡', '县', '府', '道', '路', '山', '河', '江', '湖', '海']

    # 常见朝代名称
    DYNASTIES = [

```

```

'夏', '商', '西周', '东周', '春秋', '战国', '秦', '秦朝', '汉', '汉朝', '西汉', '东汉',
'三国', '魏', '蜀', '吴', '晋', '西晋', '东晋', '南北朝', '隋', '隋朝', '唐', '唐朝',
'五代十国', '宋', '宋朝', '北宋', '南宋', '辽', '辽朝', '金', '金朝', '元', '元朝',
'明', '明朝', '清', '清朝', '民国'
]

def __init__(self, model_name="deepseek-rl:7b", known_entities=None):
    """初始化提取器 Args: model_name: 大模型名称 known_entities: 已知实体列表（从知识图谱加载） """
    self.model_name = model_name self._entity_cache = OrderedDict()
    self._known_entities = set(known_entities or [])
    print(f"已初始化大模型地名实体提取器，使用模型: {model_name}")
    if self._known_entities:
        print(f"已加载 {len(self._known_entities)} 个已知实体")

def load_known_entities(self, neo4j_db):
    """从知识图谱加载已知实体列表 Args: neo4j_db: Neo4j 数据库实例 """
    try:
        query = """ MATCH (n) RETURN DISTINCT n.name as name LIMIT 5000 """
        results = neo4j_db.graph.run(query).data()
        self._known_entities = {row['name'] for row in results if row.get('name')}
        print(f"从知识图谱加载了 {len(self._known_entities)} 个已知实体")
    except Exception as e:
        print(f"加载已知实体失败: {e}")

def _extract_by_rules(self, text):
    """基于规则的快速实体提取 Args: text: 输入文本 Returns: list: 提取的实体列表 """ entities = set()
    # 1. 匹配战争事件名称（包含特定后缀）
    for suffix in self.WAR_EVENT_SUFFIXES:
        # 匹配 "XXX 之战" 格式
        pattern = f'[-一颀]{{2,}}{re.escape(suffix)}' matches = re.findall(pattern, text)
        entities.update(matches)
    # 2. 匹配朝代名称
    for dynasty in self.DYNASTIES:
        if dynasty in text:
            entities.add(dynasty)
    # 3. 匹配已知实体（从知识图谱加载）
    for known_entity in self._known_entities:
        if len(known_entity) >= 2 and known_entity in text:
            entities.add(known_entity)
    # 4. 匹配带称号的人物（如"曹操"、"刘备"等已知实体已在步骤 3 中匹配）
    # 这里匹配 "X 将军"、"X 帝" 等格式
    for title in self.PERSON_TITLES:
        pattern = f'[-一颀]{{1,4}}{re.escape(title)}'
        matches = re.findall(pattern, text)
        # 过滤掉纯称号（如"将军"、"帝王"）
        for match in matches:
            if len(match) > len(title):

```

```

        entities.add(match)

# 5. 匹配地名（带特定后缀）
for keyword in self.PLACE_KEYWORDS:
    pattern = f'[-一颀]{{2,}}{re.escape(keyword)}' matches = re.findall(pattern, text)
    entities.update(matches)
return list(entities)

def extract_entities(self, text):
    """ 重点：从文本中提取所有实体（包括战争事件名称、人物、组织、地点） 优化策略： 1. 先尝试规则匹配（快速，不调用模型）
    2. 如果规则匹配到足够实体（>=2 个），直接返回 3. 否则调用大模型提取 Args: text: 输入文本 Returns:
    list: 提取的实体列表 """
    if not text or len(text.strip()) == 0:
        print("输入文本为空，无法提取实体")
        return []

    # 记录文本的开头部分作为示例
    text_preview = text[:100] + "..." if len(text) > 100 else text
    print(f"准备提取文本中的地名实体，文本长度：{len(text)} 字符，文本开头：'{text_preview}'")

    # 检查缓存
    cache_key = text.strip()
    cached_entities = _cache_get(self._entity_cache, cache_key)
    if cached_entities is not None:
        print(f"命中实体抽取缓存：{cached_entities}")
        return cached_entities

    # ===== 优化：先尝试规则匹配 =====
    rule_start_time = time.time()
    rule_entities = self._extract_by_rules(text)
    rule_time = time.time() - rule_start_time
    print(f"规则匹配完成，耗时：{rule_time:.4f} 秒，匹配到 {len(rule_entities)} 个实体")

    # 如果规则匹配到足够实体（>=2 个），直接返回
    if len(rule_entities) >= 2:
        print(f"规则匹配到足够实体，跳过模型调用：{rule_entities}")
        result = self._post_process_entities(rule_entities)
        _cache_set(self._entity_cache, cache_key, result)
        return result

    # ===== 规则匹配不足，调用大模型 =====
    print(f"规则匹配实体不足，调用大模型进行提取...")

    # 构建提示词
    prompt = self._build_prompt(text)

    try:
        # 记录总体开始时间

```

```

total_start_time = time.time()

# 记录模型调用开始时间
model_start_time = time.time()
print(f"调用 {self.model_name} 模型进行实体提取...")
# 调用大模型进行实体提取
response = ollama.chat( model=self.model_name, messages=[ { "role": "system",
"content": "你是一位中国历史战争事件信息抽取专家，" "擅长识别战争事件、人物、组织和地名实体。"
"不要遗漏任何实体。返回的实体应当是具体的实体名称，而不是泛指的概念。"} , { "role": "user",
"content": prompt } ],
stream=False, keep_alive="10m" )
# 记录模型调用耗时
model_time = time.time() - model_start_time
print(f"模型响应成功，耗时：{model_time:.2f}秒，"
      f"响应长度：{len(response['message']['content'])} 字符，开始解析实体...")

# 记录解析开始时间
parse_start_time = time.time()

# 从响应中解析实体
extracted_entities = self._parse_entities_from_response(response['message']['content'])

# 记录解析耗时
parse_time = time.time() - parse_start_time

# 如果解析到了实体，记录一些示例
if extracted_entities:
    entity_examples = ', '.join(extracted_entities[:5])
    entity_examples += "... " if len(extracted_entities) > 5 else ""
    print(f"解析得到 {len(extracted_entities)} 个初步实体，解析耗时：{parse_time:.2f}秒，示例：
{entity_examples}")
else:
    print(f"解析完成但未找到任何实体，解析耗时：{parse_time:.2f}秒，请检查文本内容或模型响应")

# 记录后处理开始时间
postprocess_start_time = time.time()

# 过滤和排序结果
result = self._post_process_entities(extracted_entities)

# 记录后处理耗时
postprocess_time = time.time() - postprocess_start_time

# 计算总体耗时
total_time = time.time() - total_start_time

# 记录详细的性能统计
print(f"地名实体提取完成，总耗时：{total_time:.2f}秒")

```

```

        print(f"性能指标 - 模型调用: {model_time:.2f}秒 ({model_time/total_time:.1%}), 解析: {parse_time:.2f}秒 ({parse_time/total_time:.1%}), 后处理: {postprocess_time:.2f}秒 ({postprocess_time/total_time:.1%})")

    # 记录结果统计信息
    if result:
        result_examples = ', '.join(result[:5])
        result_examples += "..." if len(result) > 5 else ""
        avg_entity_length = sum(len(entity) for entity in result) / max(1, len(result))
        print(f"实体统计 - 初始解析: {len(extracted_entities)}个, 最终有效: {len(result)}个, 过滤率: {(1 - len(result)/max(1, len(extracted_entities))):.2%}")
        print(f"实体质量 - 平均长度: {avg_entity_length:.1f}字符, 最终实体示例: {result_examples}")
        print(f"提取比率 - 每千字符实体数: {(len(result) * 1000 / max(1, len(text))):.2f}")
    else:
        print(f"警告: 后处理后没有剩余有效实体, 请检查过滤条件或原始提取结果")

    _cache_set(self._entity_cache, cache_key, result)
    return result

except Exception as e:
    error_type = type(e).__name__
    error_details = str(e)
    print(f"大模型实体提取失败: {error_type} - {error_details}")
    import traceback
    print(f"错误追踪: {traceback.format_exc()}")
    # 如果大模型调用失败, 返回空列表
    return []

def _build_prompt(self, text):
    """构建提示词, 引导模型提取所有实体""" prompt = f"""请从下面文本中提取所有实体: {text} 需要抽取的实体类型包括: 1. 战争事件名称
    2. 人物姓名 3. 国家 / 朝代 / 军队 / 政权 / 组织 4. 地点 (古地名) 要求: - 不要遗漏重要实体, 提取所有实体 请以 JSON 格式返回提取结果, 格式如下:
    ```json {{ "entities": ["赤壁之战", "曹操", "刘备", "东吴", "长江", ...] }}``` 只返回 JSON 结果, 不要有其他解释。"""
 return prompt

def _parse_entities_from_response(self, response_text):
 """从模型响应中解析实体列表"""
 try:
 # 尝试从响应中提取 JSON 部分
 json_start = response_text.find('{')
 json_end = response_text.rfind('}') + 1

 if json_start >= 0 and json_end > json_start:
 json_str = response_text[json_start:json_end]
 try:
 data = json.loads(json_str)
 if 'entities' in data and isinstance(data['entities'], list):

```

```

 print(f"成功通过 JSON 格式解析，找到实体数量: {len(data['entities'])}")
 return data['entities']
 except Exception as json_err:
 print(f"JSON 解析失败: {str(json_err)}")

如果没找到 JSON 或解析失败，尝试其他解析方法
查找列表形式
if '[' in response_text and ']' in response_text:
 list_start = response_text.find('[')
 list_end = response_text.rfind(']') + 1
 if list_start >= 0 and list_end > list_start:
 list_str = response_text[list_start:list_end]
 try:
 entities = json.loads(list_str)
 if isinstance(entities, list):
 print(f"通过列表格式解析，找到实体数量: {len(entities)}")
 return entities
 except Exception as list_err:
 print(f"列表解析失败: {str(list_err)}")

print("标准解析失败，尝试按行分割进行解析")
回退方案：按行分割并清理
lines = response_text.split('\n')
entities = []
for line in lines:
 line = line.strip()
 # 移除行首的数字、点、破折号等前缀
 line = line.lstrip('0123456789.-\'')
 line = line.strip()
 if line and len(line) >= 2:
 entities.append(line)

print(f"通过行分割解析，找到实体数量: {len(entities)}")
return entities

except Exception as e:
 print(f"解析模型响应失败: {str(e)}")
 print(f"原始响应: {response_text}")
 return []

def _post_process_entities(self, entities):
 """对提取的实体进行后处理，包括去重、过滤和排序"""
 if not entities:
 return []

 # 去重
 unique_entities = list(set(entities))
 print(f"实体去重: {len(entities)} -> {len(unique_entities)} 个")

```

```

过滤明显不是地名的实体和太短的实体
filtered_entities = [entity for entity in unique_entities
 if len(entity) >= 2 and not self._should_filter(entity)]

print(f"实体过滤: {len(unique_entities)} -> {len(filtered_entities)}个")
if len(unique_entities) > len(filtered_entities):
 filtered_out = set(unique_entities) - set(filtered_entities)
 print(f"被过滤掉的实体: {' ', ' '.join(filtered_out)}")

按长度排序（优先考虑较长的地名，通常更具体）
filtered_entities.sort(key=len, reverse=True)

返回所有有效实体，不限制数量
return filtered_entities

def _should_filter(self, entity):
 """检查实体是否应该被过滤"""
 # 过滤常见的非事件 / 非人物 / 非组织 / 非地点词汇
 non_entity_words = [

 # ===== 疑问词 / 语气词 =====
 "什么", "哪些", "如何", "为何", "为什么", "怎么", "怎样", "哪里", "谁",
 "何时", "多少", "几时", "是否", "能否", "可否",

 # ===== 泛指提问表达 =====
 "情况", "原因", "结果", "过程", "影响", "背景", "意义",
 "经过", "结局", "发展", "变化", "评价", "作用",

 # ===== 常见功能词 =====
 "是什么", "有什么", "有哪些", "怎么样", "怎么办", "属于", "位于",
 "发生", "发生在", "参与", "进行", "展开", "爆发", "结束",

 # ===== 抽象历史概念 =====
 "历史", "时期", "时代", "阶段", "当时", "后来", "之前", "之后",
 "古代", "近代", "现代", "当年", "早期", "晚期",

 # ===== 泛指地理概念（非具体地名） =====
 "地区", "地方", "区域", "位置", "地点", "城市", "省份", "国家", "行政区划", "边境", "边界", "境内", "境外",
 "内地", "中原", "南方", "北方",

 # ===== 模糊行政称谓 =====
 "县城", "府城", "郡县", "州县", "辖区",
 "首府", "都城", "京城", "国都",

 # ===== 自然地理泛词 =====
 "山川", "河流", "江河", "湖泊", "平原",
 "高原", "山地", "沙漠", "草原",

 # ===== 战争泛词（不是具体事件名）=====

```

```

 "战争", "战斗", "战役", "冲突", "交战", "进攻", "防守", "围攻", "突围", "作战", "交锋", "会战",
 # ===== 组织泛称 =====
 "军队", "部队", "兵力", "兵马", "士兵", "敌军", "友军", "联军", "朝廷", "政府", "官府", "朝廷军",
 ",
 # ===== 人物泛称 =====
 "将军", "士兵", "皇帝", "国王", "大臣", "统帅", "主帅", "元帅", "将领", "君主", "帝王",
 # ===== 系统 / 无效词 =====
 "问题", "内容", "资料", "信息", "文本", "描述", "介绍", "分析", "说明"]
检查是否是地名词汇
if entity in non_entity_words:
 return True

过滤纯数字
if entity.isdigit():
 return True

过滤只有一个字符的实体
if len(entity) <= 1:
 return True

return False

```

## 第二部分：前端核心源码

frontend/src/views/inference/index.vue

说明：智能问答页面

<!-- 智能问答页面 功能：基于大模型的历史战争知识问答 - 自然语言提问 - 展示 AI 回答和相关知识图谱 - 显示推理耗时和识别实体 - 支持多轮对话历史

API: POST/GET /api/ai/inference 参数: question(用户问题) 处理流程: 实体提取 → Neo4j 查询 → 规则推理 → 大模型生成回答 -->

```

<template> <div class="inference-container"> <!-- 左侧聊天列表 -->
 <div class="chat-sidebar" :class="{ 'sidebar-collapsed': !showSidebar }">
 <div class="sidebar-header">
 <button type="button" class="new-chat-button" @mousedown.stop @click.stop="createNewChat">
 新建聊天 </button> </div> <div class="chat-history"> <div
 v-for="(chat, index) in chatHistory" :key="index" class="chat-item"
 :class="{ 'active': currentIndex === index }" >
 <div class="chat-item-content" @click="switchChat(index)">
 <div class="chat-title">{{ chat.title || '新对话' }}</div>
 <div class="chat-time">{{ formatTime(chat.lastTime) }}</div>
 </div> <div class="chat-actions"> <lay-icon type="layui-icon-delete" color="#ff5722"
 class="delete-icon"></lay-icon> </div> </div> </div> <!-- 右侧聊天内容 -->
 <div class="chat-content"> <!-- 添加覆盖层，用于移动设备上阻止事件传播 -->
 <div class="sidebar-overlay" v-if="isMobileView && showSidebar" @click="toggleSidebar"></div>

<!-- 移除 lay-card，使用 flex 布局替代 --> <div class="chat-header"> <div class="header-left">
 <div class="fold-button" @click="toggleSidebar">

```

```

 <lay-icon :type="showSidebar ? 'layui-icon-spread-left' : 'layui-icon-shrink-right'"
color="#009688" size="16px"></lay-icon>
 </div>
 <div class="header-title">{{ currentChat.title || '新对话' }}</div>
</div> <div class="header-actions"> <!-- 添加导出按钮 --> <lay-icon type="layui-icon-export"
color="#009688" size="20px" class="export-button" title="导出对话到 Markdown" ></lay-icon>
</div> </div> <!-- 聊天区域包装器 --> <div class="message-wrapper"> <!-- 消息列表 -->
<div class="message-container" ref="messageContainer">
 <div v-if="isFirstLoad && currentChat.messages.length === 0" class="empty-chat">
 <div class="empty-icon">
 <lay-icon type="layui-icon-loading" size="60px" color="#dcdcdc"></lay-icon>
 </div>
 <div class="empty-text">正在加载聊天记录...</div>
 </div>
 <div v-else-if="currentChat.messages.length === 0" class="empty-chat">
 <div class="empty-icon">
 <lay-icon type="layui-icon-dialogue" size="60px" color="#dcdcdc"></lay-icon>
 </div>
 <div class="empty-text">开始新的对话</div> <!-- 添加快捷提示 -->
 <div class="quick-prompts"> <div class="prompt-title">常用提示:</div>
 <div class="prompt-items">
 <div class="prompt-item" v-for="(prompt, idx) in quickPrompts" :key="idx"
@click="useQuickPrompt(prompt)">
 {{ prompt }} </div> </div> </div> </div> <template v-else> <div
v-for="(message, msgIndex) in currentChat.messages" :key="msgIndex"
class="message-item"
:class="message.role === 'user' ? 'user-message' : 'ai-message'"
>
 <div class="message-avatar">
 <lay-avatar v-if="message.role === 'user'" size="40px">用户</lay-avatar>
 <lay-avatar v-else size="40px" bg-color="#009688">AI</lay-avatar>
 </div>
 <div class="message-content">
 <!-- 用户消息 -->
 <div class="message-text" v-if="message.role === 'user'">{{ message.content }}</div>

 <!-- AI 消息部分 --> <template v-else> <!-- 思考过程显示 - 移到答案上面 --> <div
v-if="message.thinking && message.thinking.length > 0" class="thinking-section"
> <div class="thinking-header"> <div class="thinking-header-left">
 思考过程 </div>
 <div class="thinking-header-right"> <div class="thinking-tag">模型思考推理</div>
 </div> </div> <div class="thinking-content">
 <div v-for="(item, index) in message.thinking" :key="index" v-
html="renderThinkingContent(item)"></div>
 </div> </div> <!-- AI 消息使用 Markdown 渲染 -->
 <div class="message-text markdown-body" v-html="renderMarkdown(message.content)"></div>

 <!-- 知识图谱可视化 -->
 <div v-if="message.role === 'assistant' && message.kgContext" class="kg-visualization"
@click.stop>

```

```

<div class="kg-header" @click.stop="toggleKgVisualization(msgIndex, $event)">
 <lay-icon :type="showKgVisualization[msgIndex] ? 'layui-icon-up' : 'layui-icon-down'"
color="#009688"></lay-icon>
 {{ showKgVisualization[msgIndex] ? '隐藏知识图谱' : '显示知识图谱' }}
</div>

<div v-if="showKgVisualization[msgIndex]" class="kg-graph-wrapper" @click.stop
@mousedown.stop @touchstart.stop>
 <div class="kg-graph-container" @click.stop @mousedown.stop @touchstart.stop>
 <!-- 知识图谱视图，可以通过引入外部图谱库如 ECharts 完善 -->
 <div class="kg-graph-placeholder" @click.stop @mousedown.stop @touchstart.stop>
 <div v-if="getKgData(message).nodes.length === 0" class="kg-empty" @click.stop>
 </div>
 </div>
 <div class="message-time">{{ formatTime(message.time) }}</div>

 <!-- 知识图谱引用信息 -->
 <div v-if="message.role === 'assistant' && message.kgContext" class="kg-reference">
 <lay-button type="text" size="sm" >
 {{ showKgInfo[msgIndex] ? '隐藏知识图谱信息' : '查看引用图谱信息' }} </lay-button>
 <div v-if="showKgInfo[msgIndex]" class="kg-context">
 <lay-line theme="green">知识图谱参考信息</lay-line> <div class="ai-notice">
 <lay-icon type="layui-icon-about" color="#FF9800"></lay-icon>
 {{ message.fromKg === false ? '知识图谱中未找到相关信息，内容由 AI 基于通用知
识生成' : '内容由 AI 基于下方知识图谱信息生成，请注意辨别信息的准确性' }}
 </div>
 <div class="kg-content-formatted markdown-body" v-
html="renderKgContextMarkdown(message.kgContext)"></div>
 </div> </div> </div> </div> </div> </div> </div> </div>
 <div v-if="loading" class="ai-message"> <div class="message-avatar">
 <lay-avatar size="40px" bg-color="#009688">AI</lay-avatar>
 </div>
 <div class="message-content thinking-bubble">
 <div class="message-text thinking-animation">思考中</div>
 </div> </div> </div> <!-- 实体卡片 - 当识别到实体且当前消息是 AI 回复时显示 -->
 <div v-if="currentChat.messages.length > 0 &&
 currentChat.messages[currentChat.messages.length-1].role === 'assistant' &&
 currentChat.messages[currentChat.messages.length-1].entities &&
 currentChat.messages[currentChat.messages.length-1].entities.length > 0 &&
 showEntityCard" class="entity-card"> <div class="entity-card-header">
 <div class="entity-card-title">
 <lay-icon type="layui-icon-location" color="#009688" size="14px"></lay-icon>
 问题中的实体
 </div>
 <div class="entity-card-close" @click="hideEntityCard">
 <lay-icon type="layui-icon-close" color="#999" size="12px"></lay-icon>
 </div> </div> <div class="entity-list"> <lay-tag
v-for="(entity, idx) in currentChat.messages[currentChat.messages.length-1].entities"
:key="idx" size="sm" theme="primary" class="entity-tag" >{{ entity }}</lay-tag>

```

```

</div> </div> </div> <!-- 输入框和按钮 - 重新样式化为底部固定样式 --> <div class="input-
container">

 <div class="input-textarea-wrapper"> <lay-textarea v-model="currentQuery"
placeholder="请输入您的问题，系统将结合知识图谱内容进行回答..." :autosize="{minRows: 1,
maxRows: 2}"

 class="compact-textarea" ref="textareaRef" ></lay-textarea>
 <div class="send-button" @click="handleQuery" :class="{ 'disabled': loading
|| !currentQuery.trim() }">
 <lay-icon type="layui-icon-up" size="16px" :color="loading || !currentQuery.trim() ? '#ccc' :
'#009688'"></lay-icon>
 </div> </div> <div class="button-group"> <div class="input-actions">
 <div class="hint">Enter 发送 / Shift + Enter 换行</div> <lay-icon
type="layui-icon-refresh" color="#888" class="clear-button" title="清空输入" ></lay-icon>
 </div> </div> </div> </div> <!-- 消息提示 -->
 <lay-layer v-model="showNodeDetailDrawer" :title="'节点关联信息'" shade="true" :area="['520px',
'88vh']">
 <div v-if="selectedKgNode" class="kg-node-detail">
 <div class="kg-node-detail-header">
 <div>
 <div class="kg-node-detail-type">{{ typeLabel(selectedKgNode.type) }}</div>
 <h3>{{ nodeDisplayName(selectedKgNode) }}</h3> </div>
 <lay-tag>{{ selectedKgNode.type || '未知类型' }}</lay-tag> </div>
 <div v-if="nodeSummaryChips.length" class="kg-node-summary">
 <div v-for="item in nodeSummaryChips" :key="item.label" class="kg-node-chip">
 {{ item.label }} {{ item.value }} </div> </div>
 <div v-if="selectedKgNode.source_text" class="kg-node-section emphasis">
 <div class="kg-node-section-title">来源原文</div>
 <div class="kg-node-long-text">{{ selectedKgNode.source_text }}</div>
 </div>

 <div v-if="selectedKgNode.Remark" class="kg-node-section">
 <div class="kg-node-section-title">补充说明</div>
 <div class="kg-node-long-text">{{ selectedKgNode.Remark }}</div>
 </div>

 <div v-if="selectedKgNode.Description" class="kg-node-section">
 <div class="kg-node-section-title">节点说明</div>
 <div class="kg-node-long-text">{{ selectedKgNode.Description }}</div>
 </div>

 <div v-if="selectedKgNode.Result" class="kg-node-section">
 <div class="kg-node-section-title">事件结果</div>
 <div class="kg-node-long-text">{{ selectedKgNode.Result }}</div>
 </div>

 <div v-if="selectedKgNode.Impact" class="kg-node-section">
 <div class="kg-node-section-title">历史影响</div>
 <div class="kg-node-long-text">{{ selectedKgNode.Impact }}</div>
 </div>

```

```

<div v-if="structuredRelationAttributes.length" class="kg-node-section">
 <div class="kg-node-section-header">
 <div class="kg-node-section-title">关联关系</div>
 {{ structuredRelationAttributes.length }} 组
 </div> <div class="kg-structured-relations"> <div
 v-for="(item, index) in structuredRelationAttributes"
 :key="'${item.relation}-${index}'" class="kg-structured-item" >
 <div class="kg-structured-head">
 <div class="kg-relation-badge">{{ item.relation }}</div>
 <div class="kg-relation-meta"> {{ item.targets.length }} 个对象
 {{ item.evidences.length }} 条证据 </div> </div>
 <div class="kg-structured-row"> <label>关系对象</label>
 <div class="kg-target-list">
 <lay-tag v-for="target in item.targets" :key="target" class="kg-target-
tag">{{ target }}</lay-tag>
 </div>
 </div>
 <div class="kg-structured-row" v-if="item.evidences.length">
 <label>证据</label>
 <div class="kg-evidence-list">
 <div v-for="evidence in item.evidences" :key="evidence" class="kg-evidence-
card">{{ evidence }}</div>
 </div>
 </div>
 </div>
</div>

```

```

import { ref, computed, watch, nextTick, onMounted, reactive, onBeforeUnmount } from 'vue';
import Http from '../api/http';
import config from '../config';
import { useUserStore } from '../store/user';
import MarkdownIt from 'markdown-it';
import hljs from 'highlight.js';
// 导入知识图谱组件
import KgGraph from './components/KgGraph.vue';
import { fieldLabel, nodeDisplayName, normalizeType, typeLabel } from '../utils/knowledge';

// 用户 store (用于获取 token) const userStore = useUserStore(); // 创建 Markdown 渲染器实例
const md = new MarkdownIt({ html: true, linkify: true, typographer: true,
highlight: function (str, lang) {
 if (lang && hljs.getLanguage(lang)) {
 try {
 return '<pre class="hljs"><code>' +
 hljs.highlight(str, { language: lang, ignoreIllegals: true }).value +
 '</code></pre>'; } catch (__) {}
 }
 return '<pre class="hljs"><code>' + md.utils.escapeHtml(str) + '</code></pre>';
 } }); // Markdown 渲染函数 function renderMarkdown(text: string): string {
 if (!text) return '';
 return md.render(text);
} interface Message { role: 'user' | 'assistant'; content: string; time: number;

```

```

kgContext?: string; // 知识图谱上下文 thinking?: string[]; // 思考过程 fromKg: boolean; // 是否来自知识图谱
entities?: string[]; // 实体列表 kg_data?: any; // 单独存储原始的 kg_data } interface Chat { id: number;
title: string; messages: Message[]; lastTime: number; } // 状态变量 const currentQuery = ref('');
const loading = ref(false);
const messageContainer = ref<HTMLElement | null>(null);
const showKgInfo = reactive<{[key: number]: boolean}>({});
const isFirstLoad = ref(true);
const isMobileView = ref(window.innerWidth < 768);
const showSidebar = ref(true);
const initialSidebarState = ref(true); // 添加变量存储初始侧边栏状态
const isSidebarLocked = ref(false); // 添加锁定变量，防止意外改变侧边栏状态
const showCopySuccess = ref(false);
const showExportSuccess = ref(false);
const isMac = navigator.platform.toUpperCase().indexOf('MAC') >= 0;

// 快捷提示 const quickPrompts = ["淝水之战的主战场在哪里？", "谁是赤壁之战的主要指挥者？", "长平之战的结果如何？"];
// 控制知识图谱可视化的显示状态 - 默认展开
const showKgVisualization = ref<Record<number, boolean>>({});

// 添加 ref 用于获取 textarea 元素 const textareaRef = ref(null); // 控制实体卡片是否显示
const showEntityCard = ref(true); const showNodeDetailDrawer = ref(false);
const selectedKgNode = ref<Record<string, any> | null>(null);
const nodeLongTextKeys = new Set(['source_text', 'Remark', 'Description', 'Result', 'Impact']);

const nodeSummaryChips = computed(() => { const node = selectedKgNode.value || {};
.chat-sidebar { position: absolute; top: 0; left: 0; height: 100%;
box-shadow: 2px 0 10px rgba(0,0,0,0.1); z-index: 1000; /* 增加 z-index 值确保在最上层 */ }
/* 添加覆盖层，阻挡其他元素的点击事件影响侧边栏 */ .sidebar-overlay { position: fixed; top: 0; left: 0;
right: 0;
bottom: 0; background-color: rgba(0, 0, 0, 0.15); z-index: 999; /* 低于侧边栏但高于其他内容 */
display: none; /* 默认隐藏 */ } /* 当侧边栏显示时显示覆盖层 */
.chat-sidebar:not(.sidebar-collapsed) + .chat-content .sidebar-overlay {
display: block; } .chat-content { width: 100%; position: relative; z-index: 1; }
/* 确保知识图谱容器不会影响侧边栏 */ .kg-graph-container { transition: none; /* 移除过渡效果，避免与侧
边栏冲突 */ }
/* 知识图谱展开时确保内容不超出右侧边界 */ .kg-graph-placeholder { max-width: 100%; overflow-x:
auto; } }
/* 添加头部左侧区域样式 */ .header-left { display: flex; align-items: center; gap: 10px; }
/* 折叠按钮样式 */ .fold-button { cursor: pointer; width: 24px; height: 24px; display: flex;
align-items: center; justify-content: center; border-radius: 4px; transition: all 0.2s; }
.fold-button:hover { background-color: #f0f9f6; } /* 修改右侧聊天内容区域 */ .chat-content { flex: 1;
min-width: 0; display: flex; flex-direction: column; height: 100%; min-height: 0;
overflow: hidden; background-color: #fff; } /* 添加聊天头部 */
from { opacity: 0; transform: translateY(10px); }
to { opacity: 1; transform: translateY(0); } } /* 思考过程样式 */ .thinking-section {
margin: 12px 0 16px 0; border-left: 3px solid #009688; position: relative; }
.thinking-header { display: flex; justify-content: space-between; align-items: center;
padding: 8px 12px; background-color: #edf7f5; border-top: 1px solid #d1e7dd;

```

```

border-right: 1px solid #d1e7dd; border-bottom: 1px solid #d1e7dd; margin-bottom: 0; }
.thinking-header-left { display: flex; align-items: center; gap: 6px; font-weight: 500;
color: #009688; } .thinking-header-right { display: flex; align-items: center; }
.thinking-tag { background-color: #009688; color: white; font-size: 12px; padding: 2px 8px;
border-radius: 4px; } .thinking-content { background-color: #f8fbfa; padding: 12px 16px;
margin-top: 0; border-right: 1px solid #d1e7dd; border-bottom: 1px solid #d1e7dd;
white-space: pre-wrap; word-break: break-word; line-height: 1.6; } .thinking-content h4 {
margin: 12px 0 8px 0; font-size: 15px; font-weight: 600; color: #333; }
.thinking-content h4:first-child { margin-top: 0; } .thinking-content h5 {
margin: 10px 0 6px 0; font-size: 14px; font-weight: 500; color: #444; }
.thinking-content .list-item { margin: 4px 0 4px 12px; position: relative; }
.thinking-content strong { font-weight: 600; color: #0a5d52; } 0% { width: 0; }
25% { width: 0.25em; } 50% { width: 0.5em; } 75% { width: 0.75em; } 100% { width: 1em; } }
/* 消息底部样式 */ .message-footer { display: flex; justify-content: space-between;
align-items: center; margin-top: 8px; } .message-actions { display: flex; gap: 8px; }
.action-icon { cursor: pointer; opacity: 0.6; transition: all 0.3s; } .action-icon:hover {
opacity: 1; transform: scale(1.2); } .message-time { font-size: 12px; color: #999; }
/* 知识图谱相关样式 */ .kg-reference { margin-left: 0 !important; width: 100%; margin-top: 12px; }
.kg-context { margin-top: 10px; padding: 16px; background-color: #f0f9f6;
border-radius: 12px; border: 1px solid #e0e0e0; animation: fadeIn 0.3s ease;
width: 100%; /* 确保占满宽度 */ box-sizing: border-box; } .ai-notice { margin: 10px 0;
padding: 8px 12px; background-color: #fff3e0; border-radius: 4px;
border-left: 3px solid #ff9800; display: flex; align-items: center; gap: 8px;
font-size: 13px; color: #e65100; } /* 知识图谱可视化样式 */ .kg-visualization { margin-top: 15px;
border-radius: 8px; background-color: #f7f9fa; overflow: hidden; position: relative;
z-index: 1; pointer-events: auto; /* 确保图谱区域有自己的点击事件 */ isolation: isolate; /* 创建新的堆叠
上下文 */
width: 100%; } .kg-graph-wrapper { height: 340px; min-height: 340px; width: 100%; }
.kg-graph-placeholder { min-height: 420px; height: 420px; } .kg-graph { min-height: 420px; }
.kg-graph-wrapper { height: 550px; min-height: 550px; width: 100%; }
.kg-graph-placeholder { min-height: 520px; height: 520px; } .kg-graph { min-height: 520px; }
/* 确保移动设备上知识图谱不会影响侧边栏 */ .kg-visualization { overflow: visible; } .kg-graph-wrapper {
padding: 10px; } .kg-graph-placeholder { max-width: 100%; overflow-x: auto; } } /* 实体卡片样式 */
.entity-card { background-color: #f7f9fa; border-radius: 6px; margin: 8px 0;
padding: 8px 12px; box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05); border: 1px solid #eee; }
.entity-card-header { display: flex; justify-content: space-between; align-items: center;
margin-bottom: 8px; } .entity-card-title { display: flex; align-items: center;
font-weight: 500; color: #009688; font-size: 13px; } .entity-card-title span {
margin-left: 4px; } .entity-card-close { cursor: pointer; opacity: 0.6; width: 18px;
height: 18px; display: flex; align-items: center; justify-content: center;
border-radius: 50%; transition: all 0.2s; } .entity-card-close:hover { opacity: 1;
background-color: #eee; } .entity-list { display: flex; flex-wrap: wrap; gap: 6px; }
.entity-tag { cursor: pointer; transition: transform 0.2s; font-size: 12px; padding: 1px 8px;
} .entity-tag:hover { transform: scale(1.05); } /* 空聊天状态 */ .empty-chat { display: flex;
0% { opacity: 0; transform: translate(-50%, -50%) scale(0.8); }
15% { opacity: 1; transform: translate(-50%, -50%) scale(1); }
80% { opacity: 1; transform: translate(-50%, -50%) scale(1); }
100% { opacity: 0; transform: translate(-50%, -50%) scale(0.8); }
} /* Markdown 样式 */ .markdown-body { line-height: 1.6; word-break: break-word; }

```

```

.markdown-body :deep(table) { border-collapse: collapse; margin-bottom: 10px; width: 100%;
overflow: auto; } .markdown-body :deep(table th), .markdown-body :deep(table td) {
padding: 6px 13px; border: 1px solid #dfe2e5; vertical-align: top; word-break: break-word; }
.markdown-body :deep(table th) { background-color: #f2f8f6; font-weight: 600; text-align: left;
color: #009688; } .markdown-body :deep(table tr) { background-color: #fff;
border-top: 1px solid #c6cbd1; } .markdown-body :deep(table tr:nth-child(2n)) {
background-color: #f6faf9; } /* 输入框相关样式 */ .clear-button { cursor: pointer; opacity: 0.6;
transition: opacity 0.3s; } .clear-button:hover { opacity: 1; } /* 特定于表格的样式 */
.markdown-body :deep(table td:nth-child(5)) { font-weight: 500; }
.markdown-body :deep(table td:last-child) { max-width: 350px; } /* 添加紧凑型输入框样式 */
.compact-textarea { font-size: 14px; line-height: 1.5; }
.compact-textarea :deep(.layui-textarea) { padding: 6px 8px;
padding-right: 45px; /* 为发送按钮留出空间 */ min-height: 22px !important; line-height: 1.5;
resize: none; border-radius: 20px; box-shadow: 0 1px 6px rgba(0,0,0,0.05);
transition: all 0.3s; }

from {
 opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: translateY(0); } }
.entity-card-title { font-size: 14px; font-weight: 600; color: #009688; } .entity-tag {
background: linear-gradient(135deg, #e0f2f1 0%, #b2dfdb 100%);
border: 1px solid rgba(0, 150, 136, 0.2); border-radius: 20px; padding: 6px 14px;
font-size: 13px; transition: all 0.3s ease; } .entity-tag:hover {
background: linear-gradient(135deg, #009688 0%, #26a69a 100%);
color: white; transform: translateY(-2px); box-shadow: 0 4px 12px rgba(0, 150, 136, 0.3); }
/* 思考过程美化 */ .thinking-section { border-left: none; border-radius: 12px; overflow: hidden;
box-shadow: 0 4px 16px rgba(0, 0, 0, 0.06); margin-bottom: 20px; } .thinking-header {
background: linear-gradient(135deg, #e3f2fd 0%, #bbdefb 100%);
border: none; padding: 12px 16px; } .thinking-header-left { color: #1565c0; font-weight: 600;
} .thinking-tag {
background: linear-gradient(135deg, #2196f3 0%, #64b5f6 100%);
box-shadow: 0 2px 8px rgba(33, 150, 243, 0.3); } .thinking-content {
background: linear-gradient(180deg, #fafafa 0%, #f5f5f5 100%);
border: 1px solid rgba(0, 0, 0, 0.05); border-top: none; padding: 16px 20px; }
/* 知识图谱可视化区域美化 */ .kg-visualization { border-radius: 16px; overflow: hidden;
box-shadow: 0 8px 32px rgba(0, 0, 0, 0.08); border: 1px solid rgba(0, 150, 136, 0.1);
margin-top: 20px; } .kg-header {
background: linear-gradient(135deg, #e0f2f1 0%, #b2dfdb 100%);
padding: 14px 20px; font-weight: 600; transition: all 0.3s ease; } .kg-header:hover {
.chat-sidebar { box-shadow: 4px 0 30px rgba(0, 0, 0, 0.15); } .message-content {
max-width: 90% !important; } .entity-card { margin: 12px; } .empty-chat { margin: 20px;
padding: 40px 24px; } } /* 加载动画美化 */ .thinking-bubble {
background: linear-gradient(135deg, #f5f7fa 0%, #e3f2fd 100%);
border: 1px solid rgba(33, 150, 243, 0.15); } .dot-animation::after { color: #2196f3; }
/* 思考中文字样式 */ .thinking-animation { color: #546e7a; font-weight: 500; }
/* 固定在主内容区内布局，避免整页滚动，只保留消息列表内部滚动。 */ .inference-container { height: 100%;
min-height: 0; }

.chat-sidebar, .chat-content, .message-wrapper { min-height: 0; } .chat-sidebar {
width: 230px; flex: 0 0 230px; } .chat-sidebar.sidebar-collapsed { width: 0 !important;
flex: 0 0 0 !important; padding: 0 !important; border-right: none !important;
overflow: hidden !important; } .chat-header { height: 44px; padding: 6px 14px; }

```

```

.sidebar-header { padding: 12px 14px; } .new-chat-button { width: 100%; height: 42px;
border: none; border-radius: 8px; background: #009688; color: #fff; font-size: 15px;
font-weight: 600; cursor: pointer; box-shadow: 0 2px 8px rgba(0, 150, 136, 0.28);
transition: all 0.2s ease; .prompt-items { grid-template-columns: 1fr; } } .kg-node-detail {
padding: 22px; display: flex; flex-direction: column; gap: 16px; } .kg-node-detail-header {
display: flex; justify-content: space-between; align-items: flex-start; gap: 12px; }
.kg-node-detail-header h3 { margin: 6px 0 0; font-size: 22px; color: #1f2937; }
.kg-node-detail-type { font-size: 12px; color: #009688; font-weight: 600;
letter-spacing: 0.08em; } .kg-node-summary { display: grid;
grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 10px; } .kg-node-chip {
padding: 12px 14px; border-radius: 12px;
background: linear-gradient(135deg, #f8fafc 0%, #eefaf8 100%);
border: 1px solid rgba(0, 150, 136, 0.12); } .kg-node-chip span { display: block;
font-size: 12px; color: #6b7280; margin-bottom: 4px; } .kg-node-chip strong { color: #1f2937;
line-height: 1.5; } .kg-node-section { border: 1px solid rgba(15, 23, 42, 0.08);
border-radius: 14px; padding: 14px 16px; background: #fff; } .kg-node-section.emphasis {
background: linear-gradient(135deg, #fffaf0 0%, #f4fffd 100%);
border-color: rgba(0, 150, 136, 0.18); } .kg-node-section-title { font-size: 14px;
font-weight: 700; color: #0f766e; margin-bottom: 10px; } .kg-node-section-header {
display: flex; align-items: center; justify-content: space-between; .kg-node-summary {
grid-template-columns: 1fr; } .kg-node-field { grid-template-columns: 1fr; gap: 4px; }
.kg-structured-row { grid-template-columns: 1fr; gap: 6px; } .kg-structured-head {
flex-direction: column; align-items: flex-start; } } </style>

```

frontend/src/views/knowledge/EntityDetail.vue

说明：实体详情页

```

<template> <div class="entity-detail-page"> <div class="hero-card"> <div>
<div class="eyebrow">实体详情页</div> <h1>{{ displayName }}</h1> </div>
 <div class="hero-actions" @click.capture="rememberGraphFocus">
 <lay-button @click="goBackToSource">返回原页面</lay-button>
 <lay-button @click="goGraphPage">查看图谱</lay-button> </div> </div>
 <div class="summary-grid"> <div class="summary-card"> 实体类型
 {{ typeLabel(entity.node?.type) }} </div> <div class="summary-card">
 关联关系
 {{ entity.quality_flags?.relation_count || 0 }}
 </div>
 <div class="summary-card">
 缺失字段
 {{ (entity.quality_flags?.missing_fields || []).length }}
 </div>
 <div class="summary-card">
 重复风险
 {{ Math.max((entity.quality_flags?.duplicate_count || 1) - 1, 0) }}
 </div> </div> <div class="content-grid"> <div class="panel"> <div class="panel-header">
<h3>核心属性</h3>
 <lay-button size="sm" @click="goRepair">进入修复工作台</lay-button>
 </div>

```

```

<div class="property-list">
 <div v-for=" [key, value] in visibleFields" :key="key" class="property-item">
 <label>{{ fieldLabel(key) }}</label> <div>{{ fieldValueLabel(key, value) }}</div>
 </div> </div>
 <div v-if="structuredRelationAttributes.length" class="structured-relations">
 <div class="structured-title">
 关联关系
 <small>{{ structuredRelationAttributes.length }} 组</small>
 </div>
 <div v-for="(item, index) in structuredRelationAttributes" :key="'${item.relation}-${index}'"
class="structured-item">
 <div class="structured-head"> <div class="relation-badge">{{ item.relation }}</div>
 <div class="relation-meta"> {{ item.targets.length }} 个对象
 {{ item.evidences.length }} 条证据 </div> </div>
 <div class="structured-row"> <label>关系对象</label> <div class="tag-list">
 <lay-tag v-for="target in item.targets" :key="target">{{ target }}</lay-tag>
 暂无关系对象
 </div> </div> <div class="structured-row"> <label>证据</label>
 <div class="evidence-list">
 <div v-for="evidence in item.evidences" :key="evidence" class="evidence-
card">{{ evidence }}</div>
 暂无证据
 </div> </div> </div> </div> </div> <div class="panel"> <div class="panel-header">
 <h3>质量提示</h3> 优先修复会影响问答和时间轴的字段 </div>
import { computed, onMounted, ref, watch } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { layer } from '@layui/layui-vue'
import { getEntityDetail } from '../..../api/module/workspace'
import { fieldLabel, fieldValueLabel, problemLabel, toEditableFields, typeLabel } from
'../..../utils/knowledge'

const route = useRoute() const router = useRouter() const entity = ref<any>({ node: {},
quality_flags: {}, relations: [], timeline_context: [], }) const displayName = computed(() => {
const node = entity.value.node || {}
 return node.EventName || node.PersonName || node.OrgName || node.geo_name || node.name || '未命名实体'
})

const visibleFields = computed(() => toEditableFields(entity.value.node || {}).filter((key) => key !==
'relations'))
const coordinateQualityText = computed(() => {
 const coord = entity.value.quality_flags?.coordinate_quality
 if (!coord) return '暂无坐标质检信息'
 const parts = [coord.is_mappable ? '可在地图显示' : '暂不可在地图显示',
`来源: ${coord.source_label || '未解析'}`,]
 if (coord.confidence) parts.push(`置信度: ${fieldValueLabel('coord_confidence', coord.confidence)}`)
 if (coord.note) parts.push(coord.note)
 return parts.join('; ')
})

const structuredRelationAttributes = computed(() => {

```

```

const rawValue = entity.value.node?.relations
if (!rawValue) return []

let relationItems: any[] = []
if (Array.isArray(rawValue)) {
 relationItems = rawValue } else if (typeof rawValue === 'string') { try {
 const parsed = JSON.parse(rawValue)
 relationItems = Array.isArray(parsed) ? parsed : [parsed]
} catch {
 return []
} } else if (typeof rawValue === 'object') { relationItems = [rawValue] }
const grouped = new Map<string, { relation: string; targets: Set<string>; evidences: Set<string> }>()

relationItems.forEach((item) => {
 if (!item || typeof item !== 'object') return

 const relation = String(item.type || item.relation || item.label || '未标注关系').trim()
 const targets = [item.to, item.target, item.object,
 ... (Array.isArray(item.targets) ? item.targets : []),
 ... (Array.isArray(item.objects) ? item.objects : []),] .flat()
 .map((value) => String(value || '').trim()) .filter(Boolean) const evidences = [
 item.evidence,
 ... (Array.isArray(item.evidences) ? item.evidences : []),
] .flat() .map((value) => String(value || '').trim()) .filter(Boolean)
 if (!grouped.has(relation)) {
 grouped.set(relation, { relation, targets: new Set<string>(),
 evidences: new Set<string>(), }) } const current = grouped.get(relation)!
 targets.forEach((target) => current.targets.add(target))
 .summary-grid, .content-grid { grid-template-columns: 1fr 1fr; } } .entity-detail-page {
padding: 14px; } .hero-card, .summary-grid, .content-grid { grid-template-columns: 1fr;
display: grid; } .structured-row { grid-template-columns: 1fr; } .structured-head {
align-items: flex-start; flex-direction: column; } .hero-actions { margin-top: 8px; } }
</style>

```